

南 京 师 范 大 学

电气与自动化工程学院

《嵌入式系统原理及应用》课内实验报告



专 业： _____ 自动化 _____

班 级： _____ 212309 _____ 学 号： _____ 21230933 _____

姓 名： _____ 赵学文 _____

填写日期： _____ 2026 年 6 月 13 日 _____

电气与自动化工程学院 制

一、实验任务

1. 完成开发板串口接入与 U-Boot 交互环境建立。
2. 完成查询类命令验证，熟悉 U-Boot 中板级信息、版本信息和环境变量信息的查看方式。
3. 完成环境变量修改、保存、新建和删除操作，掌握环境变量在启动流程中的控制作用。
4. 完成 DRAM 读、写、填充、复制和比较实验，理解 U-Boot 对内存空间的直接操作能力。
5. 完成 EMMC 与 ext4 文件系统访问实验，掌握镜像文件的查询、装载和写入方法。
6. 完成 Linux 启动命令验证，掌握从镜像装载到内核启动的基本路径。
7. 完成 ums、run、mii info 等扩展命令验证，理解 U-Boot 在维护和调试中的辅助作用。

二、实验条件

2.1 平台条件

- 启动介质：板载 EMMC
- 主机系统：Ubuntu Linux
- 串口终端工具：picocom

2.2 连接条件

- 开发板已完成上电准备。
- 串口线已连接至主机。
- 主机能够识别串口设备节点。
- 开发板中已经存在可供读取的 uImage 和设备树文件。

2.3 实验基础

本次课内实验建立在 Ubuntu 系统安装已完成的前提下，重点不再放在主机环境部署，而放在 BootLoader 阶段命令体系的使用、观察与分析上。实验对象

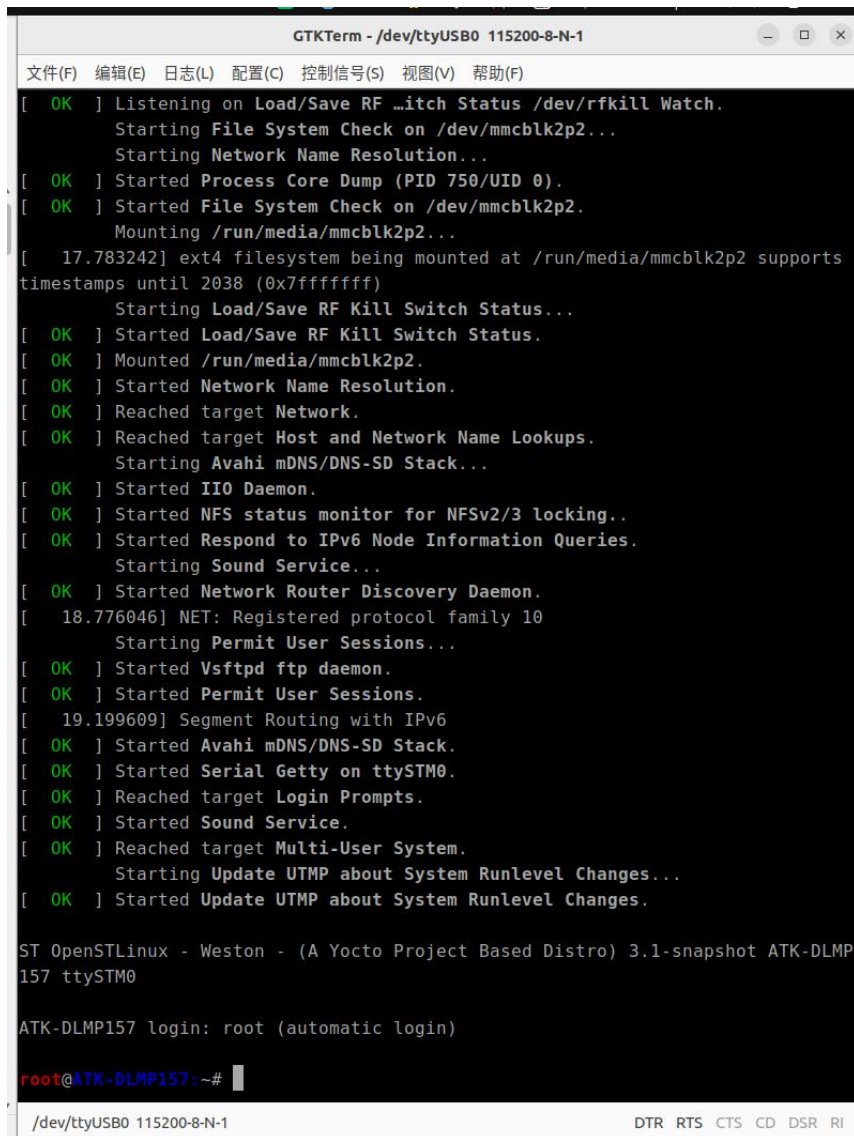
并非抽象的“命令列表”，而是启动过程中的各个关键节点。每条命令都对应一个系统状态切面，命令执行结果共同拼接出 U-Boot 的实际工作路径。

U-Boot 在嵌入式 Linux 启动过程中处于承上启下的位置。一方面，它需要完成时钟、串口、DRAM 等底层硬件准备；另一方面，它又要负责把镜像、设备树和启动参数组织好，再把控制权交给 Linux 内核。因此，只要 U-Boot 阶段的状态判断出现偏差，后续启动结论就会随之失真。本次实验的价值，正是把这一阶段拆成多个可观察、可验证、可分析的任务块。

三、实验过程记录

3.1 任务一：建立 U-Boot 交互环境

实验开始后，首先需要确认串口通道稳定，并保证系统能够在倒计时期间进入交互式命令行。开发板上电后，串口终端中能够看到 U-Boot 启动日志，说明串口控制器、线缆连接和主机端终端工具均工作正常。



```
GTKTerm - /dev/ttyUSB0 115200-8-N-1
文件(F) 编辑(E) 日志(L) 配置(C) 控制信号(S) 视图(V) 帮助(H)

[ OK ] Listening on Load/Save RF ...itch Status /dev/rfkill Watch.
Starting File System Check on /dev/mmcblk2p2...
Starting Network Name Resolution...
[ OK ] Started Process Core Dump (PID 750/UID 0).
[ OK ] Started File System Check on /dev/mmcblk2p2.
Mounting /run/media/mmcblk2p2...
[ 17.783242] ext4 filesystem being mounted at /run/media/mmcblk2p2 supports
timestamps until 2038 (0x7fffffff)
Starting Load/Save RF Kill Switch Status...
[ OK ] Started Load/Save RF Kill Switch Status.
[ OK ] Mounted /run/media/mmcblk2p2.
[ OK ] Started Network Name Resolution.
[ OK ] Reached target Network.
[ OK ] Reached target Host and Network Name Lookups.
Starting Avahi mDNS/DNS-SD Stack...
[ OK ] Started IIO Daemon.
[ OK ] Started NFS status monitor for NFSv2/3 locking..
[ OK ] Started Respond to IPv6 Node Information Queries.
Starting Sound Service...
[ OK ] Started Network Router Discovery Daemon.
[ 18.776046] NET: Registered protocol family 10
Starting Permit User Sessions...
[ OK ] Started Vsftpd ftp daemon.
[ OK ] Started Permit User Sessions.
[ 19.199609] Segment Routing with IPv6
[ OK ] Started Avahi mDNS/DNS-SD Stack.
[ OK ] Started Serial Getty on ttySTM0.
[ OK ] Reached target Login Prompts.
[ OK ] Started Sound Service.
[ OK ] Reached target Multi-User System.
Starting Update UTMP about System Runlevel Changes...
[ OK ] Started Update UTMP about System Runlevel Changes.

ST OpenSTLinux - Weston - (A Yocto Project Based Distro) 3.1-snapshot ATK-DLMP
157 ttySTM0

ATK-DLMP157 login: root (automatic login)

root@ATK-DLMP157:~#
```

图 3-1 串口连接与启动界面

实验使用 `picocom -b 115200 -d 8 -p n -f n /dev/ttyUSB0` 建立串口连接。该命令给出了完整的串口工作参数，其中 115200 对应波特率，-d 8 对应 8 位数据位，-p n 表示无校验，-f n 表示无流控。这类参数虽然在表面上看只是串口工具的启动格式，但实际上直接决定日志能否被正确解码。若参数不匹配，串口现象往往不是“启动失败”，而是乱码、丢字或完全无输出。

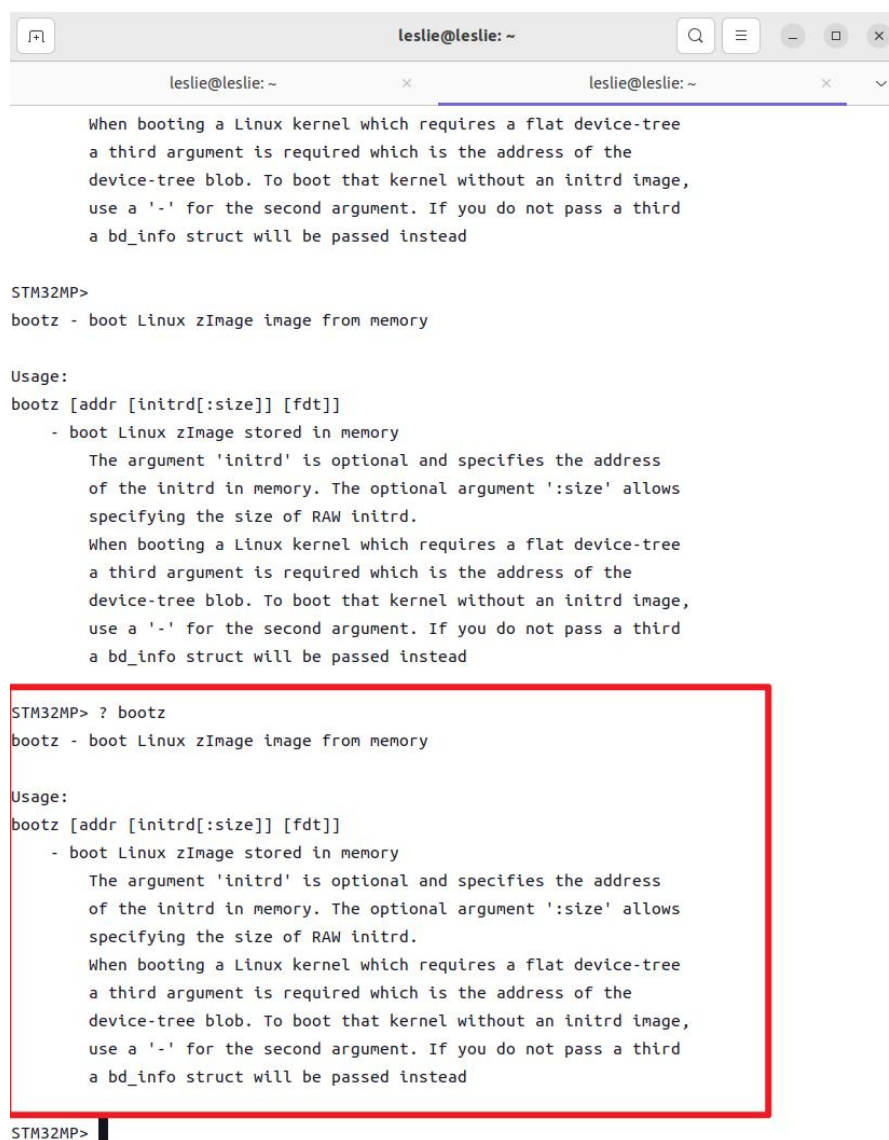
看到启动日志后，在倒计时结束前按键，即可中断默认启动流程并进入 STM32MP> 提示符。能够进入提示符，说明当前平台已经具备人工干预启动过程的条件。这一点在课程实验中非常关键，因为后续所有命令操作都建立在交互环境已经接管启动流程的基础上。若不能进入命令行，就无法对环境变量、内存状态和镜像装载过程进行逐项验证。

从实验角度看，这一步所验证的不只是“串口能通”，而是“U-Boot 交互环境已建立”。串口链路建立成功说明主机与开发板之间的观察通道已经打开；进入提示符则说明启动流程可以从自动执行切换到人工控制。只有在这两个条件同时满足时，后续实验链才真正开始。

3.2 任务二：完成查询类命令验证

查询类命令承担的是“观察当前状态”的任务。与修改类、写入类命令不同，这些命令不会直接改变系统配置，而是把系统当前已经形成的状态展示出来。对 U-Boot 来说，这一类命令的价值在于建立分析前提。

3.2.1 查看 bootz 帮助信息



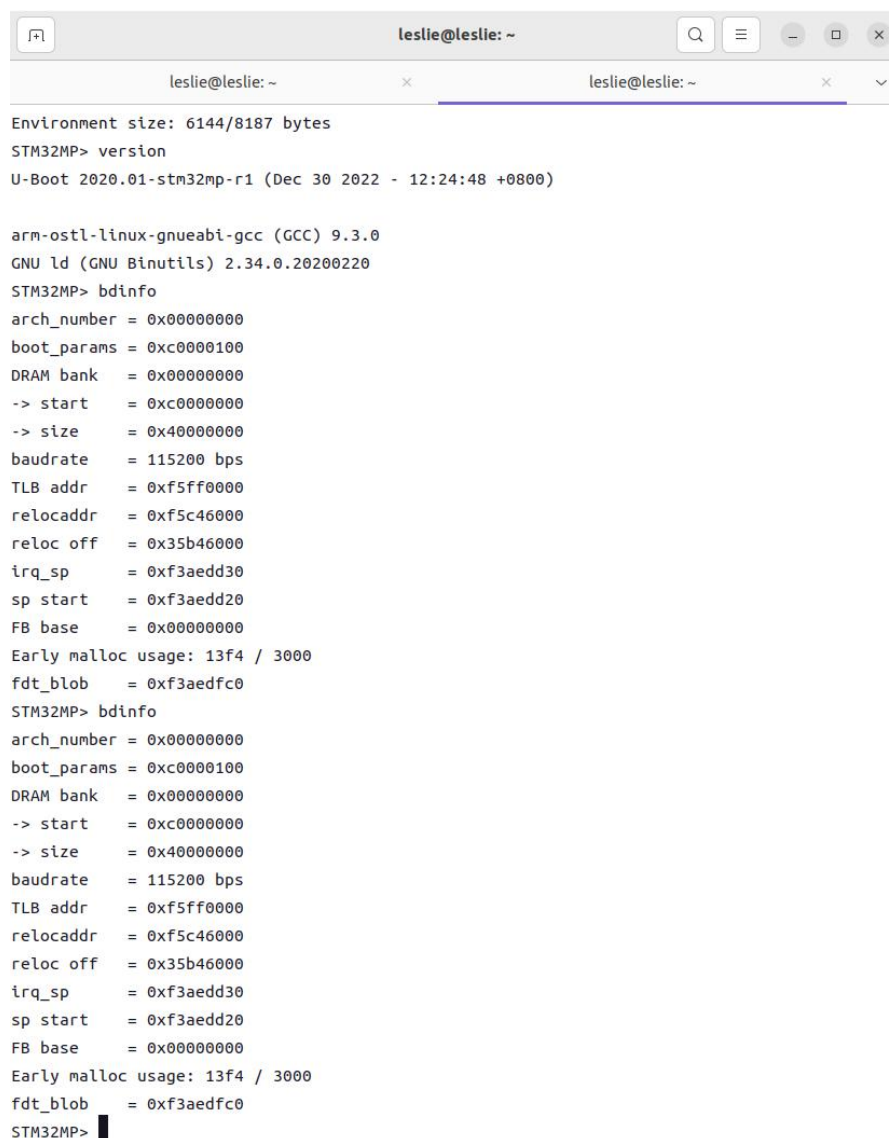
```
leslie@leslie: ~  
leslie@leslie: ~  
leslie@leslie: ~  
When booting a Linux kernel which requires a flat device-tree  
a third argument is required which is the address of the  
device-tree blob. To boot that kernel without an initrd image,  
use a '-' for the second argument. If you do not pass a third  
a bd_info struct will be passed instead  
  
STM32MP>  
bootz - boot Linux zImage image from memory  
  
Usage:  
bootz [addr [initrd[:size]] [fdt]]  
- boot Linux zImage stored in memory  
  The argument 'initrd' is optional and specifies the address  
  of the initrd in memory. The optional argument ':size' allows  
  specifying the size of RAW initrd.  
  When booting a Linux kernel which requires a flat device-tree  
  a third argument is required which is the address of the  
  device-tree blob. To boot that kernel without an initrd image,  
  use a '-' for the second argument. If you do not pass a third  
  a bd_info struct will be passed instead  
  
STM32MP> ? bootz  
bootz - boot Linux zImage image from memory  
  
Usage:  
bootz [addr [initrd[:size]] [fdt]]  
- boot Linux zImage stored in memory  
  The argument 'initrd' is optional and specifies the address  
  of the initrd in memory. The optional argument ':size' allows  
  specifying the size of RAW initrd.  
  When booting a Linux kernel which requires a flat device-tree  
  a third argument is required which is the address of the  
  device-tree blob. To boot that kernel without an initrd image,  
  use a '-' for the second argument. If you do not pass a third  
  a bd_info struct will be passed instead  
  
STM32MP>
```

图 3-2 bootz 命令帮助信息

帮助命令验证的是 U-Boot 对命令语法的内建说明能力。bootz 属于 zImage 启动命令，虽然本实验后续主要以 bootm 配合 uImage 为主，但保留 bootz 的帮助界面，能够说明 U-Boot 本身具备多种镜像格式的处理接口。帮助信息中给出的命令格式，不只是“说明书”，也是判断当前版本是否支持某条命令的重要依据。

对于底层调试而言，先查帮助再执行命令是一种非常有价值的习惯。原因并不只是防止语法写错，更重要的是能够在当前运行环境内部确认命令是否存在、参数顺序是否变化、适用对象是否匹配。不同版本 U-Boot 在命令支持范围上可能存在差异，因此帮助查询既服务于学习，也服务于版本适配判断。

3.2.2 查看板级信息



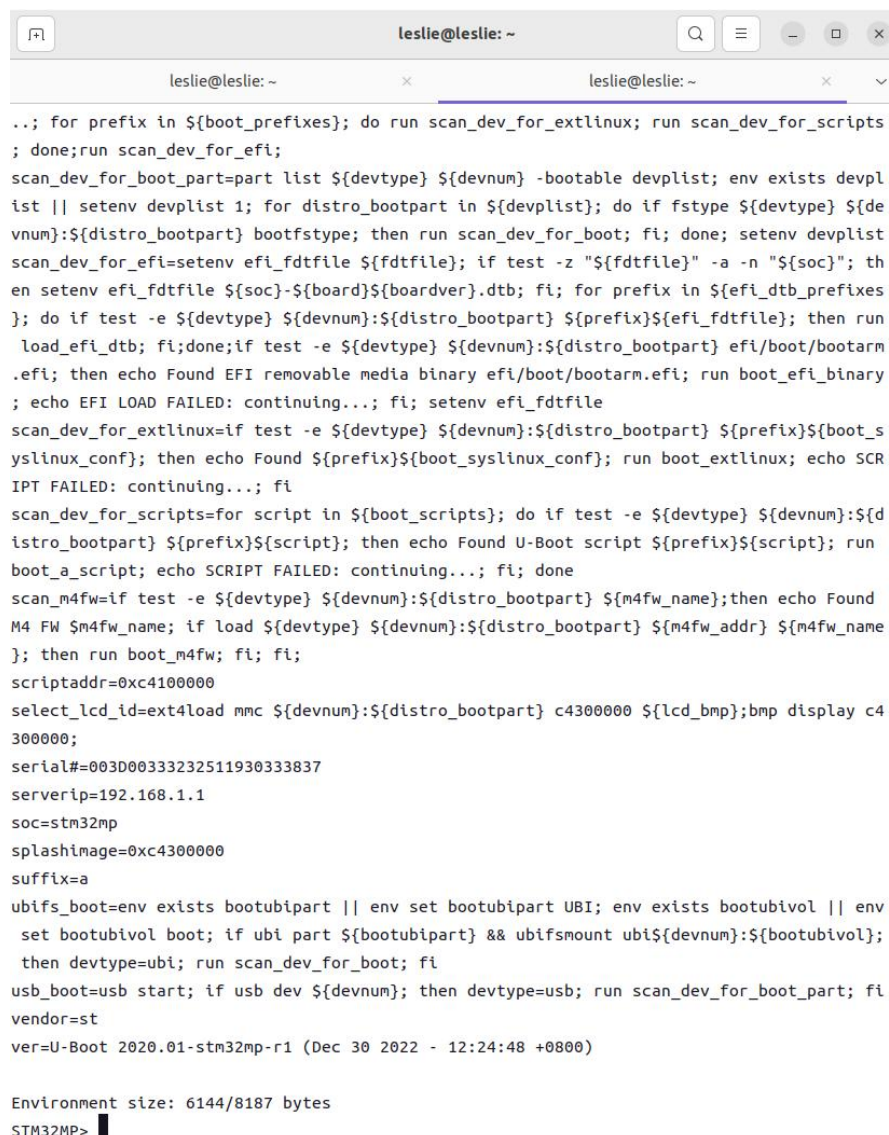
```
leslie@leslie: ~  
leslie@leslie: ~  
Environment size: 6144/8187 bytes  
STM32MP> version  
U-Boot 2020.01-stm32mp-r1 (Dec 30 2022 - 12:24:48 +0800)  
  
arm-ostl-linux-gnueabi-gcc (GCC) 9.3.0  
GNU ld (GNU Binutils) 2.34.0.20200220  
STM32MP> bdinfo  
arch_number = 0x00000000  
boot_params = 0xc0000100  
DRAM bank   = 0x00000000  
-> start     = 0xc0000000  
-> size      = 0x40000000  
baudrate    = 115200 bps  
TLB addr    = 0xf5ff0000  
relocaddr   = 0xf5c46000  
reloc off   = 0x35b46000  
irq_sp      = 0xf3aedd30  
sp start    = 0xf3aedd20  
FB base     = 0x00000000  
Early malloc usage: 13f4 / 3000  
fdt_blob    = 0xf3aedfc0  
STM32MP> bdinfo  
arch_number = 0x00000000  
boot_params = 0xc0000100  
DRAM bank   = 0x00000000  
-> start     = 0xc0000000  
-> size      = 0x40000000  
baudrate    = 115200 bps  
TLB addr    = 0xf5ff0000  
relocaddr   = 0xf5c46000  
reloc off   = 0x35b46000  
irq_sp      = 0xf3aedd30  
sp start    = 0xf3aedd20  
FB base     = 0x00000000  
Early malloc usage: 13f4 / 3000  
fdt_blob    = 0xf3aedfc0  
STM32MP>
```

图 3-3 bdinfo 命令运行结果

bdinfo 输出的是当前板级运行状态中的核心信息，包括 DRAM 起始地址与大小、启动参数地址、串口波特率、堆栈指针位置等。该命令的直接意义在于提供一份板级状态快照。后续所有涉及地址的命令，例如 **md**、**mw**、**ext4load**、**bootm**，都应建立在对地址布局已有大致判断的前提上。

从实验观察来看，**bdinfo** 的存在使“平台正在什么状态下运行”不再停留在猜测层面。若 DRAM 初始化失败，很多地址类命令本身就难以正常运行；若串口波特率配置不一致，则终端日志也不会稳定显示。也就是说，**bdinfo** 并不是一个单纯展示信息的命令，而是在为后续命令链提供背景条件确认。

3.2.3 查看环境变量



```
leslie@leslie: ~  
..; for prefix in ${boot_prefixes}; do run scan_dev_for_extlinux; run scan_dev_for_scripts  
; done; run scan_dev_for_efi;  
scan_dev_for_boot_part=part list ${devtype} ${devnum} -bootable devplist; env exists devpl  
ist || setenv devplist 1; for distro_bootpart in ${devplist}; do if fstype ${devtype} ${de  
vnum}:${distro_bootpart} bootfstype; then run scan_dev_for_boot; fi; done; setenv devplist  
scan_dev_for_efi=setenv efi_fdtfile ${fdtfile}; if test -z "${fdtfile}" -a -n "${soc}"; th  
en setenv efi_fdtfile ${soc}-${board}${boardver}.dtb; fi; for prefix in ${efi_dtb_prefixes  
}; do if test -e ${devtype} ${devnum}:${distro_bootpart} ${prefix}${efi_fdtfile}; then run  
load_efi_dtb; fi; done; if test -e ${devtype} ${devnum}:${distro_bootpart} efi/boot/bootarm  
.efi; then echo Found EFI removable media binary efi/boot/bootarm.efi; run boot_efi_binary  
; echo EFI LOAD FAILED: continuing...; fi; setenv efi_fdtfile  
scan_dev_for_extlinux=if test -e ${devtype} ${devnum}:${distro_bootpart} ${prefix}${boot_s  
yslinux_conf}; then echo Found ${prefix}${boot_syslinux_conf}; run boot_extlinux; echo SCR  
IPT FAILED: continuing...; fi  
scan_dev_for_scripts=for script in ${boot_scripts}; do if test -e ${devtype} ${devnum}:${d  
istro_bootpart} ${prefix}${script}; then echo Found U-Boot script ${prefix}${script}; run  
boot_a_script; echo SCRIPT FAILED: continuing...; fi; done  
scan_m4fw=if test -e ${devtype} ${devnum}:${distro_bootpart} ${m4fw_name}; then echo Found  
M4 FW ${m4fw_name}; if load ${devtype} ${devnum}:${distro_bootpart} ${m4fw_addr} ${m4fw_name  
}; then run boot_m4fw; fi; fi;  
scriptaddr=0xc4100000  
select_lcd_id=ext4load mmc ${devnum}:${distro_bootpart} c4300000 ${lcd_bmp}; bmp display c4  
300000;  
serial#=003D00333232511930333837  
serverip=192.168.1.1  
soc=stm32mp  
splashimage=0xc4300000  
suffix=a  
ubifs_boot=env exists bootubipart || env set bootubipart UBI; env exists bootubivol || env  
set bootubivol boot; if ubi part ${bootubipart} && ubifs mount ubi${devnum}:${bootubivol};  
then devtype=ubi; run scan_dev_for_boot; fi  
usb_boot=usb start; if usb dev ${devnum}; then devtype=usb; run scan_dev_for_boot_part; fi  
vendor=st  
ver=U-Boot 2020.01-stm32mp-r1 (Dec 30 2022 - 12:24:48 +0800)  
  
Environment size: 6144/8187 bytes  
STM32MP>
```

图 3-4 printenv 命令运行结果

printenv 负责把环境变量完整展示出来。环境变量是 U-Boot 启动行为的控制中心，bootdelay、bootcmd、bootargs、baudrate、ipaddr 等字段都直接影响后续启动现象。通过 printenv 可以看到系统并不是在一个“空白状态”下启动，而是已经带着一组默认参数、默认脚本和默认策略运行。

这一点非常重要。很多启动现象并不是命令即时生成的，而是环境变量事先定义好的。例如自动启动是否存在倒计时、倒计时结束后执行哪组命令、内核启动时带哪些参数，这些都不是临时发生的，而是变量预先约定好的。若不看 printenv，就很难真正理解系统为什么会按某种方式启动。

3.2.4 查看版本信息

图 3-5 version 命令运行结果

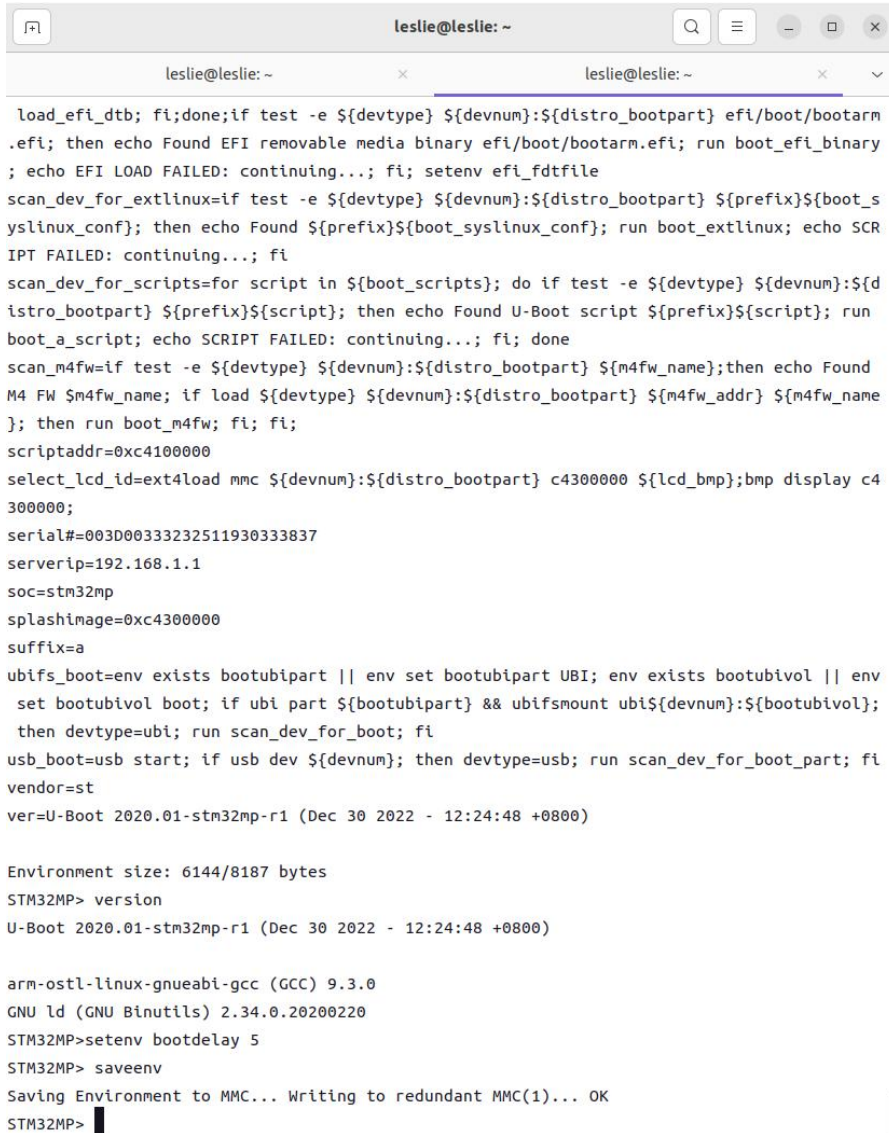
版本信息是参考手册与当前实验平台之间建立对应关系的关键。命令能否使用、语法是否一致、默认环境变量是否相同，在一定程度上都与版本有关。通过 `version` 可确认当前使用的 U-Boot 版本与参考资料大体一致，从而提高后续实验分析的可靠性。

版本检查还有一个更深层的意义，即明确实验材料的适用边界。若平台版本与文档版本差异过大，则不能机械地把文档里的每一条命令照搬到当前环境中，而应优先考虑版本适配性。这种意识对后续进行驱动开发和系统移植时同样重要。

3.3 任务三：完成环境变量操作

环境变量实验是 U-Boot 命令使用中最能体现“配置能力”的部分。U-Boot 并不是把所有启动流程都写死在代码里，而是把大量关键行为通过环境变量进行外部组织。修改环境变量，实质上就是修改启动策略。

3.3.1 修改 bootdelay



```
leslie@leslie: ~  
leslie@leslie: ~  
load_efi_dtb; fi;done;if test -e ${devtype} ${devnum}:${distro_bootpart} efi/boot/bootarm  
.efi; then echo Found EFI removable media binary efi/boot/bootarm.efi; run boot_efi_binary  
; echo EFI LOAD FAILED: continuing...; fi; setenv efi_fdtfile  
scan_dev_for_extlinux=if test -e ${devtype} ${devnum}:${distro_bootpart} ${prefix}${boot_s  
yslinux_conf}; then echo Found ${prefix}${boot_syslinux_conf}; run boot_extlinux; echo SCR  
IPT FAILED: continuing...; fi  
scan_dev_for_scripts=for script in ${boot_scripts}; do if test -e ${devtype} ${devnum}:${d  
istro_bootpart} ${prefix}${script}; then echo Found U-Boot script ${prefix}${script}; run  
boot_a_script; echo SCRIPT FAILED: continuing...; fi; done  
scan_m4fw=if test -e ${devtype} ${devnum}:${distro_bootpart} ${m4fw_name};then echo Found  
M4 FW ${m4fw_name}; if load ${devtype} ${devnum}:${distro_bootpart} ${m4fw_addr} ${m4fw_name  
}; then run boot_m4fw; fi; fi;  
scriptaddr=0xc4100000  
select_lcd_id=ext4load mmc ${devnum}:${distro_bootpart} c4300000 ${lcd_bmp};bmp display c4  
300000;  
serial#=003D00333232511930333837  
serverip=192.168.1.1  
soc=stm32mp  
splashimage=0xc4300000  
suffix=a  
ubifs_boot=env exists bootubipart || env set bootubipart UBI; env exists bootubivol || env  
set bootubivol boot; if ubi part ${bootubipart} && ubismount ubi${devnum}:${bootubivol};  
then devtype=ubi; run scan_dev_for_boot; fi  
usb_boot=usb start; if usb dev ${devnum}; then devtype=usb; run scan_dev_for_boot_part; fi  
vendor=st  
ver=U-Boot 2020.01-stm32mp-r1 (Dec 30 2022 - 12:24:48 +0800)  
  
Environment size: 6144/8187 bytes  
STM32MP> version  
U-Boot 2020.01-stm32mp-r1 (Dec 30 2022 - 12:24:48 +0800)  
  
arm-ostl-linux-gnueabi-gcc (GCC) 9.3.0  
GNU ld (GNU Binutils) 2.34.0.20200220  
STM32MP>setenv bootdelay 5  
STM32MP> saveenv  
Saving Environment to MMC... Writing to redundant MMC(1)... OK  
STM32MP>
```

图 3-6 修改 bootdelay 的过程

通过 `setenv bootdelay 5` 可以把倒计时修改为 5，通过 `saveenv` 则把该变化保存到外部存储中。这里最关键的 concept 不是“命令输入成功”，而是“变量修改发生在 DRAM 副本中，保存之后才能变成持久配置”。`setenv` 处理的是当前会话中的运行时副本，`saveenv` 才负责把结果写回持久化环境区。

环境变量机制决定了实验分析不能只看当前终端里的回显结果。若只执行 `setenv` 而不执行 `saveenv`，当前命令行里虽然已经能看到新值，但复位之后系统仍然会从外部存储中重新加载旧值。因此，对环境变量实验的判断必须包含“修改”“保存”“重启”“复核”四个动作，缺少任何一个，结论都不完整。

3.3.2 观察倒计时变化



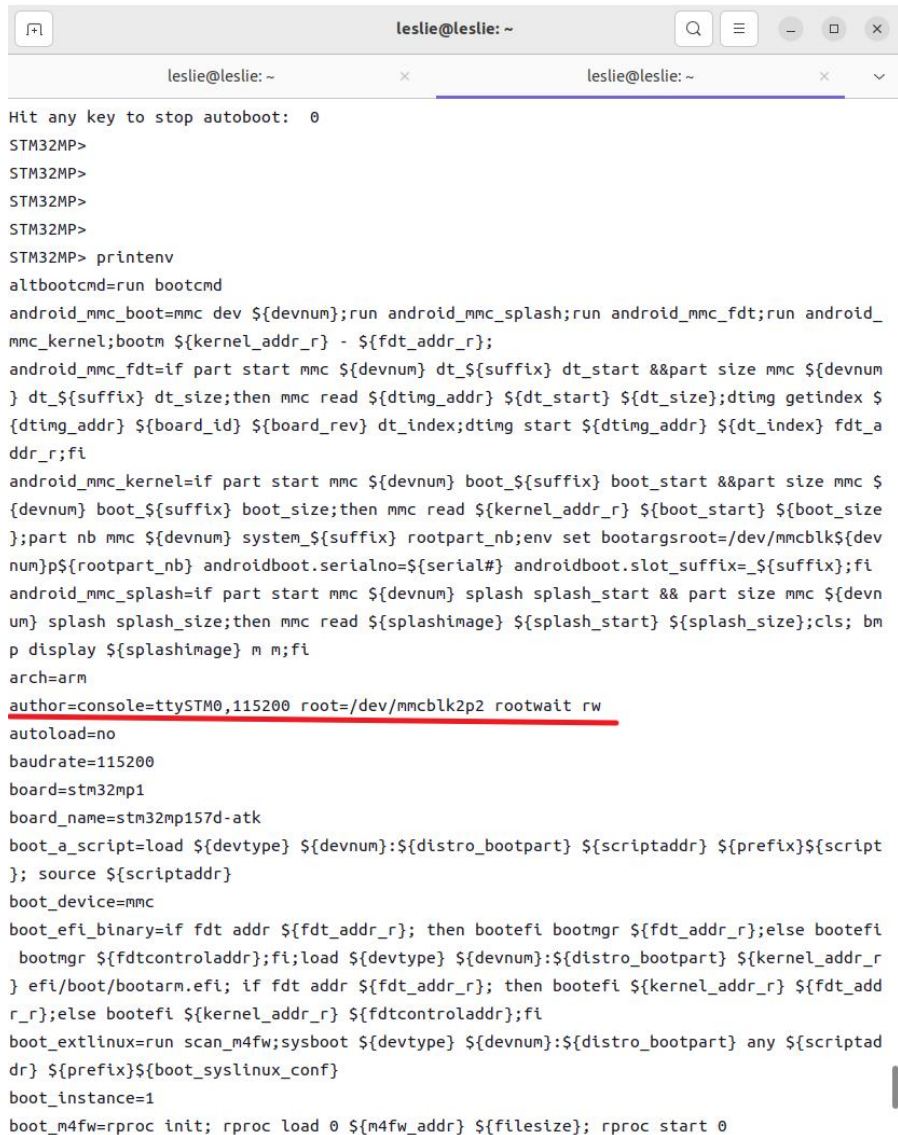
```
leslie@leslie: ~  
leslie@leslie: ~  
INFO: Cannot find st,stm32mp1 node in DT  
ERROR: Inconsistent OPP settings found in DT, ignored.  
NOTICE: SP_MIN: v2.2-r1.0(debug):21665c8  
NOTICE: SP_MIN: Built : 04:33:26, Sep 7 2022  
INFO: ARM GICv2 driver initialized  
INFO: stm32mp IWDG1 (12): Secure  
INFO: ETZPC: CRYP1 (9) could be non secure  
INFO: SP_MIN: Initializing runtime services  
INFO: SP_MIN: Preparing exit to normal world  
  
U-Boot 2020.01-stm32mp-r1 (Dec 30 2022 - 12:24:48 +0800)  
  
CPU: STM32MP157DAA Rev.Z  
Model: STMicroelectronics STM32MP157D eval daughter  
Board: stm32mp1 in trusted mode (st,stm32mp157d-atk)  
DRAM: 1 GiB  
Clocks:  
- MPU : 800 MHz  
- MCU : 208.878 MHz  
- AXI : 266.500 MHz  
- PER : 24 MHz  
- DDR : 533 MHz  
WDT: Started with servicing (32s timeout)  
NAND: 0 MiB  
MMC: STM32 SD/MMC: 0, STM32 SD/MMC: 1  
Loading Environment from MMC... OK  
In: serial  
Out: serial  
Err: serial  
invalid MAC address in OTP 00:00:00:00:00:00  
Net:  
Error: ethernet@5800a000 address not set.  
No ethernet found.  
  
lcd_id = 07  
Hit any key to stop autoboot: 5
```

图 3-7 倒计时变为 5 秒

重新启动后，启动界面的倒计时已经变为 5 秒，这说明新配置已经被重新加载，变量持久化成功。与单纯查看环境变量值相比，倒计时变化属于更直观的现象级验证，因为它直接体现在每次上电过程之中，而不仅仅停留在命令打印结果中。

这一步的重要性在于把“变量值发生变化”与“启动行为发生变化”对应起来。只有当系统行为也发生了改变，才说明环境变量已经真正参与到启动流程中，而不是只在某一轮命令行里暂时被改写过。

3.3.3 新建变量



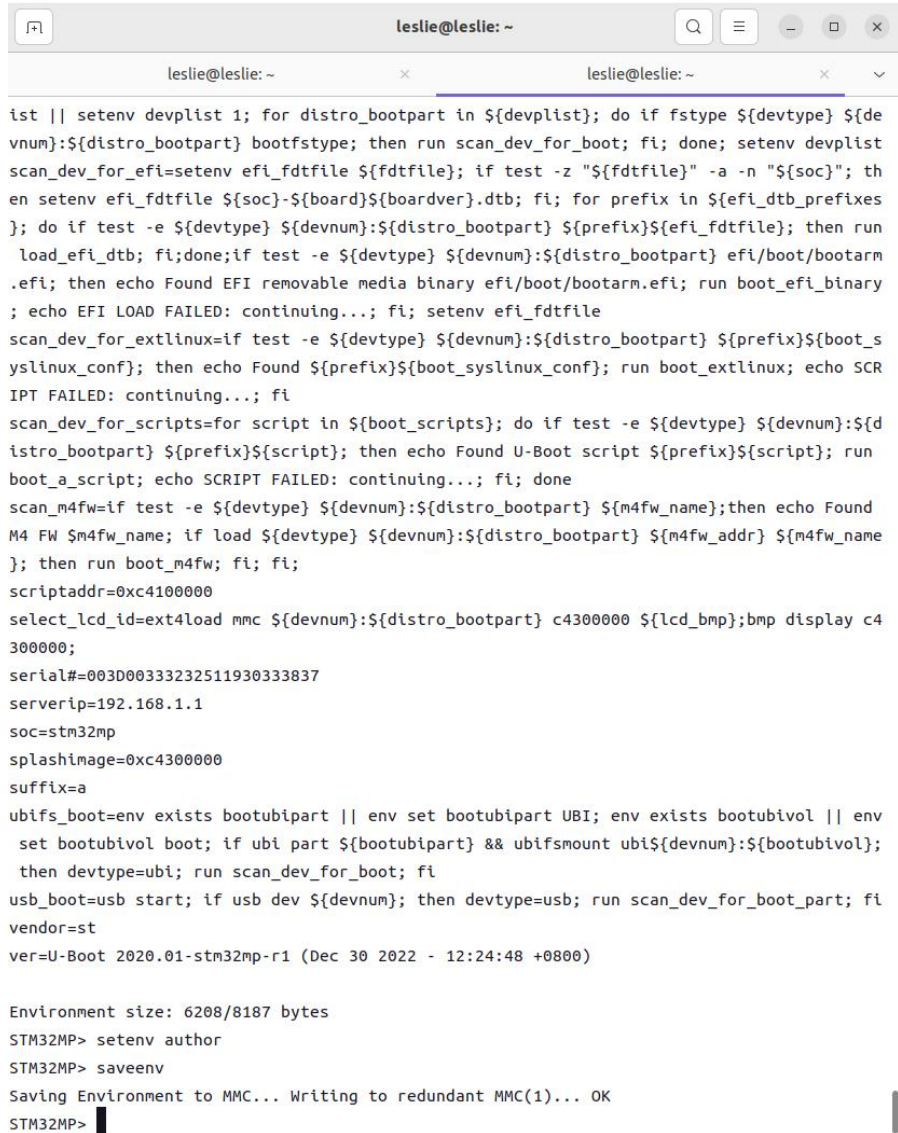
```
leslie@leslie: ~
Hit any key to stop autoboot: 0
STM32MP>
STM32MP>
STM32MP>
STM32MP>
STM32MP> printenv
altbootcmd=run bootcmd
android_mmc_boot=mmc dev ${devnum};run android_mmc_splash;run android_mmc_fdt;run android_
mmc_kernel;bootm ${kernel_addr_r} - ${fdt_addr_r};
android_mmc_fdt=if part start mmc ${devnum} dt_${suffix} dt_start &&part size mmc ${devnum
} dt_${suffix} dt_size;then mmc read ${dting_addr} ${dt_start} ${dt_size};dting getindex $
{dting_addr} ${board_id} ${board_rev} dt_index;dting start ${dting_addr} ${dt_index} fdt_a
ddr_r;fi
android_mmc_kernel=if part start mmc ${devnum} boot_${suffix} boot_start &&part size mmc $
{devnum} boot_${suffix} boot_size;then mmc read ${kernel_addr_r} ${boot_start} ${boot_size
};part nb mmc ${devnum} system_${suffix} rootpart_nb;env set bootargsroot=/dev/mmcblk${dev
num}p${rootpart_nb} androidboot.serialno=${serial#} androidboot.slot_suffix=${suffix};fi
android_mmc_splash=if part start mmc ${devnum} splash splash_start && part size mmc ${devn
um} splash splash_size;then mmc read ${splashimage} ${splash_start} ${splash_size};cls;bm
p display ${splashimage} m m;fi
arch=arm
author=console=ttySTM0,115200 root=/dev/mmcblk2p2 rootwait rw
autoload=no
baudrate=115200
board=stm32mp1
board_name=stm32mp157d-atk
boot_a_script=load ${devtype} ${devnum}:${distro_bootpart} ${scriptaddr} ${prefix}${script
}; source ${scriptaddr}
boot_device=mmc
boot_efi_binary=if fdt addr ${fdt_addr_r}; then bootefi bootmgr ${fdt_addr_r};else bootefi
bootmgr ${fdtcontroladdr};fi;load ${devtype} ${devnum}:${distro_bootpart} ${kernel_addr_r
} efi/boot/bootarm.efi; if fdt addr ${fdt_addr_r}; then bootefi ${kernel_addr_r} ${fdt_add
r_r};else bootefi ${kernel_addr_r} ${fdtcontroladdr};fi
boot_extlinux=run scan_m4fw;sysboot ${devtype} ${devnum}:${distro_bootpart} any ${scriptad
dr} ${prefix}${boot_syslinux_conf}
boot_instance=1
boot_m4fw=rproc init; rproc load 0 ${m4fw_addr} ${filesize}; rproc start 0
```

图 3-8 新建 author 环境变量

新建 author 变量所使用的内容，是一段带有多个字段的参数字符串。该步骤的重点不在变量名，而在带空格字符串的组织方式。由于变量值中包含多个以空格分隔的字段，因此必须使用单引号整体包裹，否则 U-Boot 会把这些字段分拆为多个独立参数。

这类规则在实际使用中非常常见。bootargs、网络脚本、自定义启动序列往往都不是单一词项，而是由若干字段拼接而成。只要对带空格字符串的传递方式理解不清，就很容易在后续更复杂的变量配置中出现逻辑错误。通过新建变量这一步，可以把字符串组织规则提前做成一个清晰的实验现象。

3.3.4 删除变量



```
ist || setenv devplist 1; for distro_bootpart in ${devplist}; do if fstype ${devtype} ${devnum}:${distro_bootpart} bootfstype; then run scan_dev_for_boot; fi; done; setenv devplist
scan_dev_for_efi=setenv efi_fdtfile ${fdtfile}; if test -z "${fdtfile}" -a -n "${soc}"; th
en setenv efi_fdtfile ${soc}-${board}${boardver}.dtb; fi; for prefix in ${efi_dtb_prefixes
}; do if test -e ${devtype} ${devnum}:${distro_bootpart} ${prefix}${efi_fdtfile}; then run
load_efi_dtb; fi;done;if test -e ${devtype} ${devnum}:${distro_bootpart} efi/boot/bootarm
.efi; then echo Found EFI removable media binary efi/boot/bootarm.efi; run boot_efi_binary
; echo EFI LOAD FAILED: continuing...; fi; setenv efi_fdtfile
scan_dev_for_extlinux=if test -e ${devtype} ${devnum}:${distro_bootpart} ${prefix}${boot_s
yslinux_conf}; then echo Found ${prefix}${boot_syslinux_conf}; run boot_extlinux; echo SCR
IPT FAILED: continuing...; fi
scan_dev_for_scripts=for script in ${boot_scripts}; do if test -e ${devtype} ${devnum}:${d
istro_bootpart} ${prefix}${script}; then echo Found U-Boot script ${prefix}${script}; run
boot_a_script; echo SCRIPT FAILED: continuing...; fi; done
scan_m4fw=if test -e ${devtype} ${devnum}:${distro_bootpart} ${m4fw_name};then echo Found
M4 FW ${m4fw_name}; if load ${devtype} ${devnum}:${distro_bootpart} ${m4fw_addr} ${m4fw_name
}; then run boot_m4fw; fi; fi;
scriptaddr=0xc4100000
select_lcd_id=ext4load mmc ${devnum}:${distro_bootpart} c4300000 ${lcd_bmp};bmp display c4
300000;
serial#=003D00333232511930333837
serverip=192.168.1.1
soc=stm32mp
splashimage=0xc4300000
suffix=a
ubifs_boot=env exists bootubipart || env set bootubipart UBI; env exists bootubivol || env
set bootubivol boot; if ubi part ${bootubipart} && ubismount ubi${devnum}:${bootubivol};
then devtype=ubi; run scan_dev_for_boot; fi
usb_boot=usb start; if usb dev ${devnum}; then devtype=usb; run scan_dev_for_boot_part; fi
vendor=st
ver=U-Boot 2020.01-stm32mp-r1 (Dec 30 2022 - 12:24:48 +0800)

Environment size: 6208/8187 bytes
STM32MP> setenv author
STM32MP> saveenv
Saving Environment to MMC... Writing to redundant MMC(1)... OK
STM32MP>
```

图 3-9 删除 author 环境变量

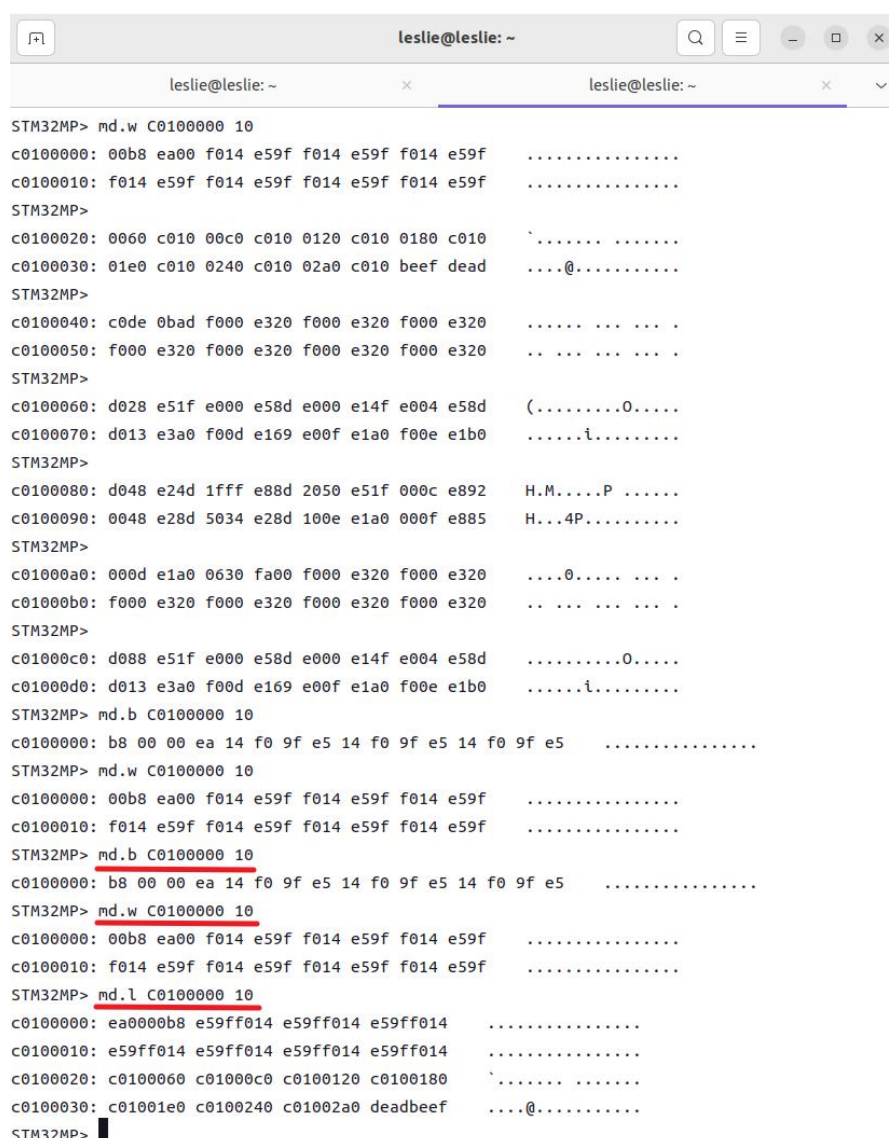
删除变量的方式仍然是使用 `setenv`，但变量名后不再跟随具体内容。该操作说明 U-Boot 在环境变量管理接口上保持了统一设计：修改变量靠 `setenv`，新建变量靠 `setenv`，删除变量也靠 `setenv`。不同动作之间并不依赖不同命令，而是依赖传入值的不同。

这一点在命令体系学习中很重要。接口统一意味着学习路径更短，但同时也要求使用者真正理解“给变量赋值”和“把变量置空”之间的区别。会改、会增、会删，环境变量部分才算真正闭环。

3.4 任务四：完成 DRAM 操作验证

内存操作是 U-Boot 最具底层特征的一类命令。镜像文件最终会进入 DRAM，设备树也会进入 DRAM，很多启动参数与中间数据同样落在 DRAM 中。因此，只要把 DRAM 操作能力掌握清楚，后续分析镜像装载和启动现象就会更有抓手。

3.4.1 使用 md 读取内存



```
leslie@leslie: ~  
leslie@leslie: ~  
STM32MP> md.w C0100000 10  
c0100000: 00b8 ea00 f014 e59f f014 e59f f014 e59f .....  
c0100010: f014 e59f f014 e59f f014 e59f f014 e59f .....  
STM32MP>  
c0100020: 0060 c010 00c0 c010 0120 c010 0180 c010 .....  
c0100030: 01e0 c010 0240 c010 02a0 c010 beef dead ....@.....  
STM32MP>  
c0100040: c0de 0bad f000 e320 f000 e320 f000 e320 .....  
c0100050: f000 e320 f000 e320 f000 e320 f000 e320 .....  
STM32MP>  
c0100060: d028 e51f e000 e58d e000 e14f e004 e58d .....0.....  
c0100070: d013 e3a0 f00d e169 e00f e1a0 f00e e1b0 .....i.....  
STM32MP>  
c0100080: d048 e24d 1fff e88d 2050 e51f 000c e892 H.M.....P .....  
c0100090: 0048 e28d 5034 e28d 100e e1a0 000f e885 H...4P.....  
STM32MP>  
c01000a0: 000d e1a0 0630 fa00 f000 e320 f000 e320 ....0.....  
c01000b0: f000 e320 f000 e320 f000 e320 f000 e320 .....  
STM32MP>  
c01000c0: d088 e51f e000 e58d e000 e14f e004 e58d .....0.....  
c01000d0: d013 e3a0 f00d e169 e00f e1a0 f00e e1b0 .....i.....  
STM32MP> md.b C0100000 10  
c0100000: b8 00 00 ea 14 f0 9f e5 14 f0 9f e5 14 f0 9f e5 .....  
STM32MP> md.w C0100000 10  
c0100000: 00b8 ea00 f014 e59f f014 e59f f014 e59f .....  
c0100010: f014 e59f f014 e59f f014 e59f f014 e59f .....  
STM32MP> md.b C0100000 10  
c0100000: b8 00 00 ea 14 f0 9f e5 14 f0 9f e5 14 f0 9f e5 .....  
STM32MP> md.w C0100000 10  
c0100000: 00b8 ea00 f014 e59f f014 e59f f014 e59f .....  
c0100010: f014 e59f f014 e59f f014 e59f f014 e59f .....  
STM32MP> md.l C0100000 10  
c0100000: ea0000b8 e59ff014 e59ff014 e59ff014 .....  
c0100010: e59ff014 e59ff014 e59ff014 e59ff014 .....  
c0100020: c0100060 c01000c0 c0100120 c0100180 .....  
c0100030: c01001e0 c0100240 c01002a0 deadbeef ....@.....  
STM32MP>
```

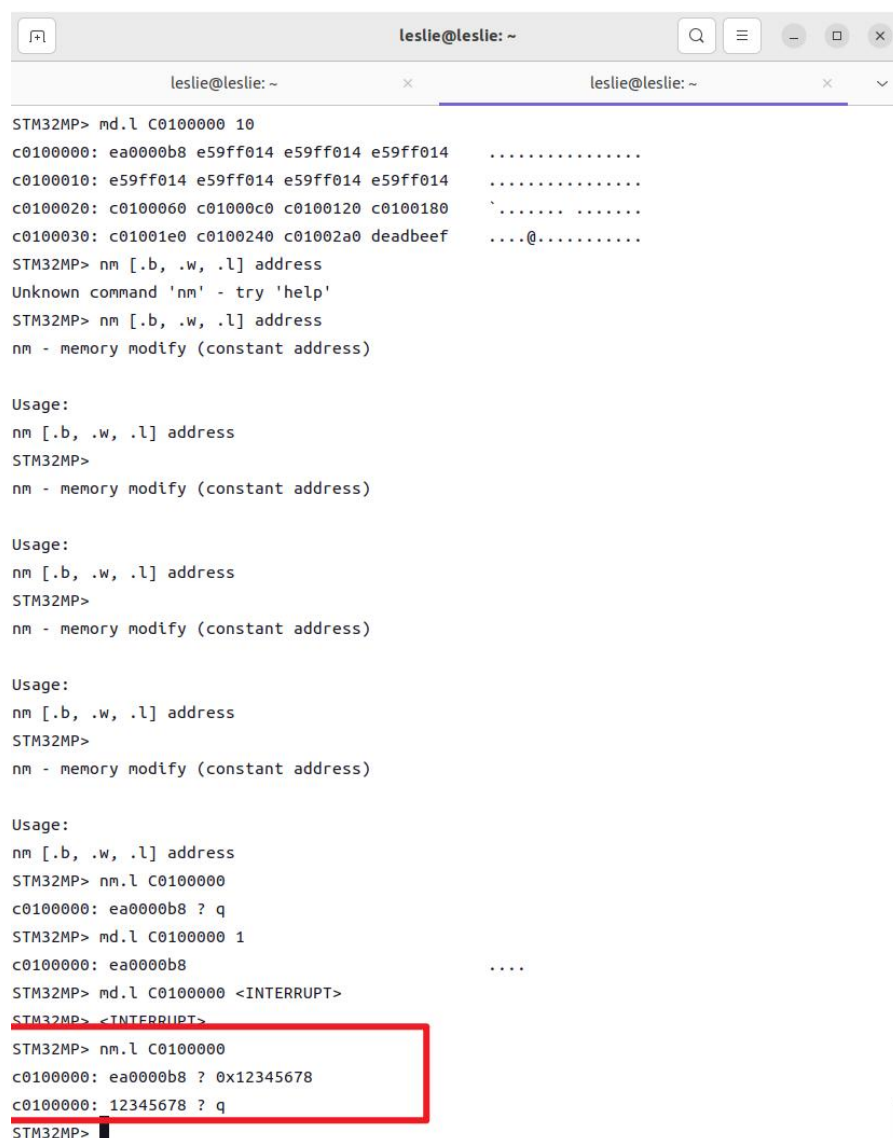
图 3-10 md 命令读取内存

md 负责显示指定地址开始的一段内容。b、w、l 分别对应字节、半字和字三种宽度。长度参数并不是总字节数，而是读取对象个数。这个规则若理解

不清，实验时就很容易把“读取数量”和“读取字节规模”混为一谈，最终导致结果判断失真。

md 的作用不仅是“看内存”，更在于把抽象地址空间变成可直接观察的数据区域。后续镜像装载后，是否已进入指定地址、数据模式是否发生变化、某段区域是否被覆盖，都可以借助 md 做初步判断。它是 U-Boot 中最基础也最常用的观察手段之一。

3.4.2 使用 nm 修改单地址内容



```
leslie@leslie: ~
STM32MP> md.l C0100000 10
c0100000: ea0000b8 e59ff014 e59ff014 e59ff014 .....
c0100010: e59ff014 e59ff014 e59ff014 e59ff014 .....
c0100020: c0100060 c01000c0 c0100120 c0100180 .....
c0100030: c01001e0 c0100240 c01002a0 deadbeef ....@.....
STM32MP> nm [.b, .w, .l] address
Unknown command 'nm' - try 'help'
STM32MP> nm [.b, .w, .l] address
nm - memory modify (constant address)

Usage:
nm [.b, .w, .l] address
STM32MP>
nm - memory modify (constant address)

Usage:
nm [.b, .w, .l] address
STM32MP>
nm - memory modify (constant address)

Usage:
nm [.b, .w, .l] address
STM32MP>
nm - memory modify (constant address)

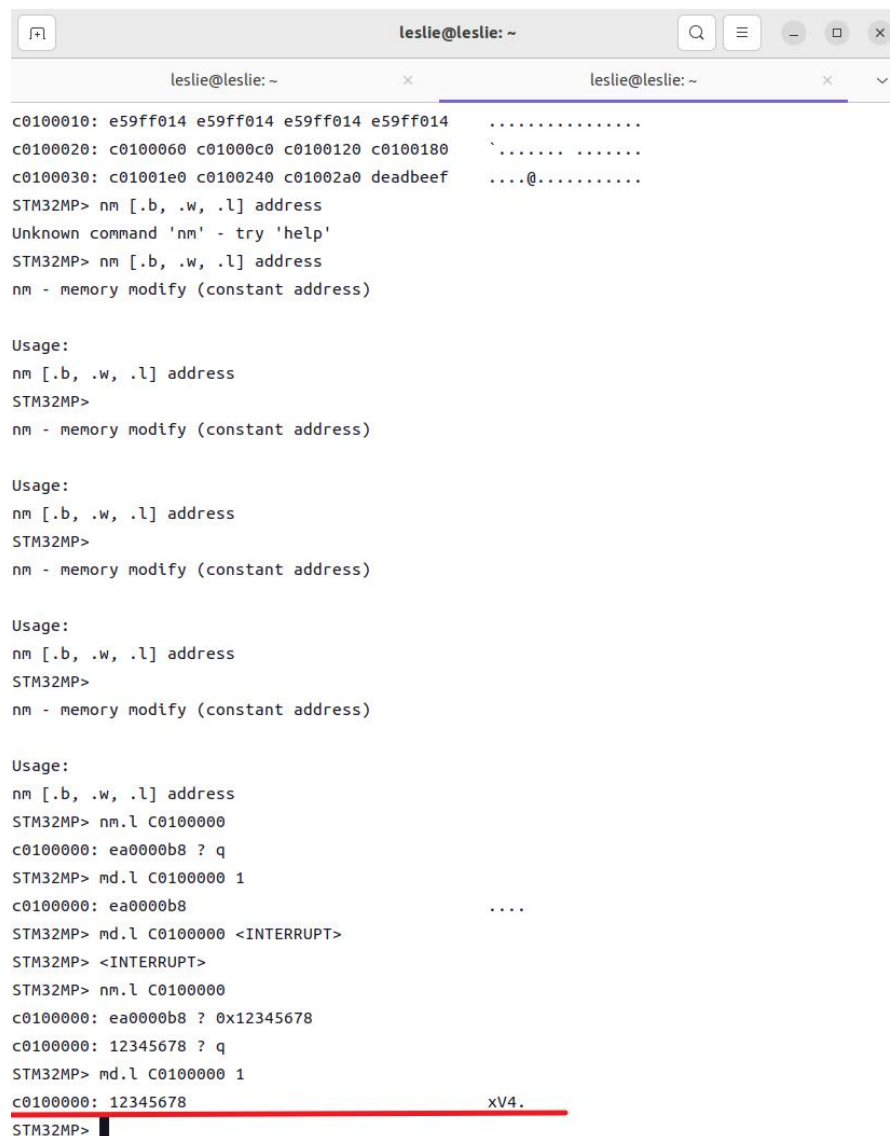
Usage:
nm [.b, .w, .l] address
STM32MP> nm.l C0100000
c0100000: ea0000b8 ? q
STM32MP> md.l C0100000 1
c0100000: ea0000b8 ....
STM32MP> md.l C0100000 <INTERRUPT>
STM32MP> <INTERRUPT>
STM32MP> nm.l C0100000
c0100000: ea0000b8 ? 0x12345678
c0100000: 12345678 ? q
STM32MP>
```

图 3-11 nm 命令修改单地址数据

nm 用于修改单个地址处的内容。该命令适合做“点修改”实验，也适合在某些关键地址上快速验证内存是否可写。与批量写入不同，nm 更接近一种试探性操作：先改一个点，再观察这个点是否真的变化。

对单地址修改的掌握，意味着对内存空间的操控已经不再停留在读取层面，而是进入了“可干预状态”。在底层调试中，这种能力非常重要，因为很多时候需要先人为修改一个小范围数据，再观察系统后续反应。

3.4.3 再次读取验证结果



```
leslie@leslie: ~  
leslie@leslie: ~  
c0100010: e59ff014 e59ff014 e59ff014 e59ff014 .....  
c0100020: c0100060 c01000c0 c0100120 c0100180 .....  
c0100030: c01001e0 c0100240 c01002a0 deadbeef ....@.....  
STM32MP> nm [.b, .w, .l] address  
Unknown command 'nm' - try 'help'  
STM32MP> nm [.b, .w, .l] address  
nm - memory modify (constant address)  
  
Usage:  
nm [.b, .w, .l] address  
STM32MP>  
nm - memory modify (constant address)  
  
Usage:  
nm [.b, .w, .l] address  
STM32MP>  
nm - memory modify (constant address)  
  
Usage:  
nm [.b, .w, .l] address  
STM32MP>  
nm - memory modify (constant address)  
  
Usage:  
nm [.b, .w, .l] address  
STM32MP> nm.l C0100000  
c0100000: ea0000b8 ? q  
STM32MP> md.l C0100000 1  
c0100000: ea0000b8 ....  
STM32MP> md.l C0100000 <INTERRUPT>  
STM32MP> <INTERRUPT>  
STM32MP> nm.l C0100000  
c0100000: ea0000b8 ? 0x12345678  
c0100000: 12345678 ? q  
STM32MP> md.l C0100000 1  
c0100000: 12345678 xV4.  
STM32MP>
```

图 3-12 修改结果的再次验证

再次使用 `md` 读取原地址后，可以看到修改结果已经写入目标位置。写操作之后紧跟读操作，是底层调试里最基本的闭环逻辑。否则，命令只是被执行了，但是否真正作用到预定地址上，仍然没有被证实。

这一过程体现出内存类实验与环境变量类实验的共同特点：写入动作本身不足以构成结论，必须通过后续验证动作补足证据。区别只在于，环境变量靠重启后的系统现象验证，内存写入则靠回读结果验证。

3.4.4 使用 mm 连续修改内存



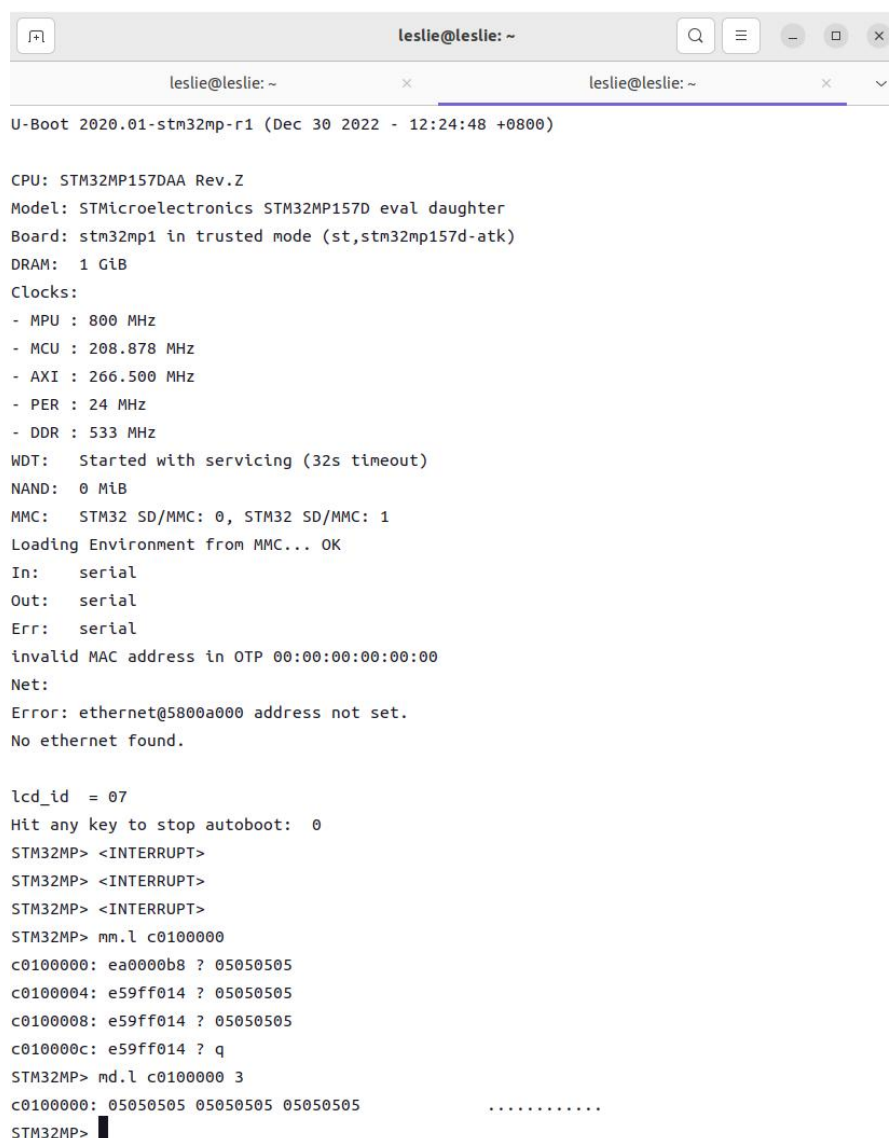
```
leslie@leslie: ~  
leslie@leslie: ~  
U-Boot 2020.01-stm32mp-r1 (Dec 30 2022 - 12:24:48 +0800)  
  
CPU: STM32MP157DAA Rev.Z  
Model: STMicroelectronics STM32MP157D eval daughter  
Board: stm32mp1 in trusted mode (st,stm32mp157d-atk)  
DRAM: 1 GiB  
Clocks:  
- MPU : 800 MHz  
- MCU : 208.878 MHz  
- AXI : 266.500 MHz  
- PER : 24 MHz  
- DDR : 533 MHz  
WDT: Started with servicing (32s timeout)  
NAND: 0 MiB  
MMC: STM32 SD/MMC: 0, STM32 SD/MMC: 1  
Loading Environment from MMC... OK  
In: serial  
Out: serial  
Err: serial  
invalid MAC address in OTP 00:00:00:00:00:00  
Net:  
Error: ethernet@5800a000 address not set.  
No ethernet found.  
  
lcd_id = 07  
Hit any key to stop autoboot: 0  
STM32MP> <INTERRUPT>  
STM32MP> <INTERRUPT>  
STM32MP> <INTERRUPT>  
STM32MP> mm.l c0100000  
c0100000: ea0000b8 ? 05050505  
c0100004: e59ff014 ? 05050505  
c0100008: e59ff014 ? 05050505  
c010000c: e59ff014 ? q  
STM32MP>
```

图 3-13 mm 命令交互界面

mm 与 nm 相似，但每次输入完成后地址会自动递增，因此更适合连续修改相邻地址的内容。若 nm 强调的是“单点控制”，那么 mm 强调的就是“顺序编辑”。这两者虽然都属于修改类命令，但对应的使用场景并不相同。

在实验逻辑中，把 nm 放在前、mm 放在后是合理的。先掌握单点修改，再掌握连续修改，能够让命令难度和数据范围逐步扩大，而不会一开始就把连续写入的风险引入进来。

3.4.5 查看 mm 修改效果

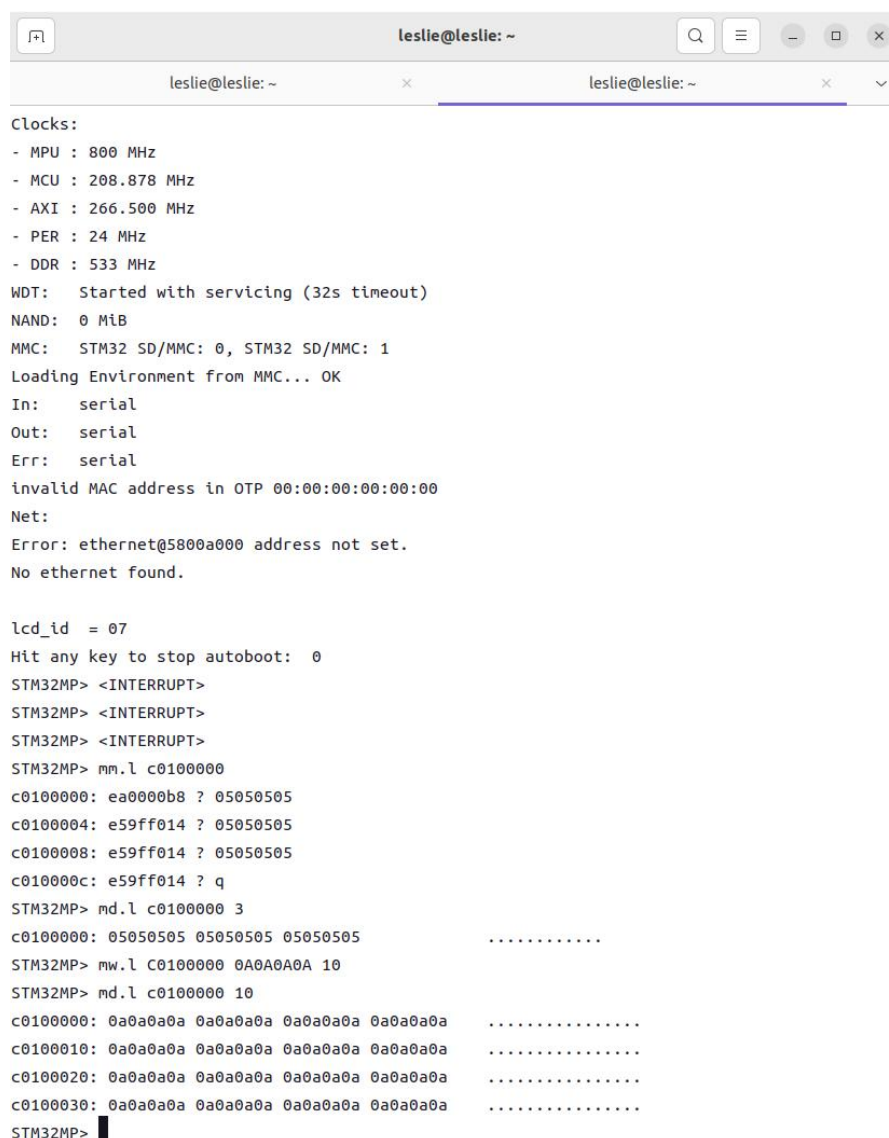


```
leslie@leslie: ~  
leslie@leslie: ~  
leslie@leslie: ~  
U-Boot 2020.01-stm32mp-r1 (Dec 30 2022 - 12:24:48 +0800)  
  
CPU: STM32MP157DAA Rev.Z  
Model: STMicroelectronics STM32MP157D eval daughter  
Board: stm32mp1 in trusted mode (st,stm32mp157d-atk)  
DRAM: 1 GiB  
Clocks:  
- MPU : 800 MHz  
- MCU : 208.878 MHz  
- AXI : 266.500 MHz  
- PER : 24 MHz  
- DDR : 533 MHz  
WDT: Started with servicing (32s timeout)  
NAND: 0 MiB  
MMC: STM32 SD/MMC: 0, STM32 SD/MMC: 1  
Loading Environment from MMC... OK  
In: serial  
Out: serial  
Err: serial  
invalid MAC address in OTP 00:00:00:00:00:00  
Net:  
Error: ethernet@5800a000 address not set.  
No ethernet found.  
  
lcd_id = 07  
Hit any key to stop autoboot: 0  
STM32MP> <INTERRUPT>  
STM32MP> <INTERRUPT>  
STM32MP> <INTERRUPT>  
STM32MP> mm.l c0100000  
c0100000: ea0000b8 ? 05050505  
c0100004: e59ff014 ? 05050505  
c0100008: e59ff014 ? 05050505  
c010000c: e59ff014 ? q  
STM32MP> md.l c0100000 3  
c0100000: 05050505 05050505 05050505 .....  
STM32MP>
```

图 3-14 mm 修改后的读取结果

从回读结果可以看到连续地址内容已经按输入顺序被逐项改写，这说明地址自增特性工作正常。连续修改之所以必须配合读取结果一起分析，是因为它比单地址修改更容易产生范围偏移，一旦不回读，就无法确认修改是否恰好落在目标区域内。

3.4.6 使用 mw 批量填充内存



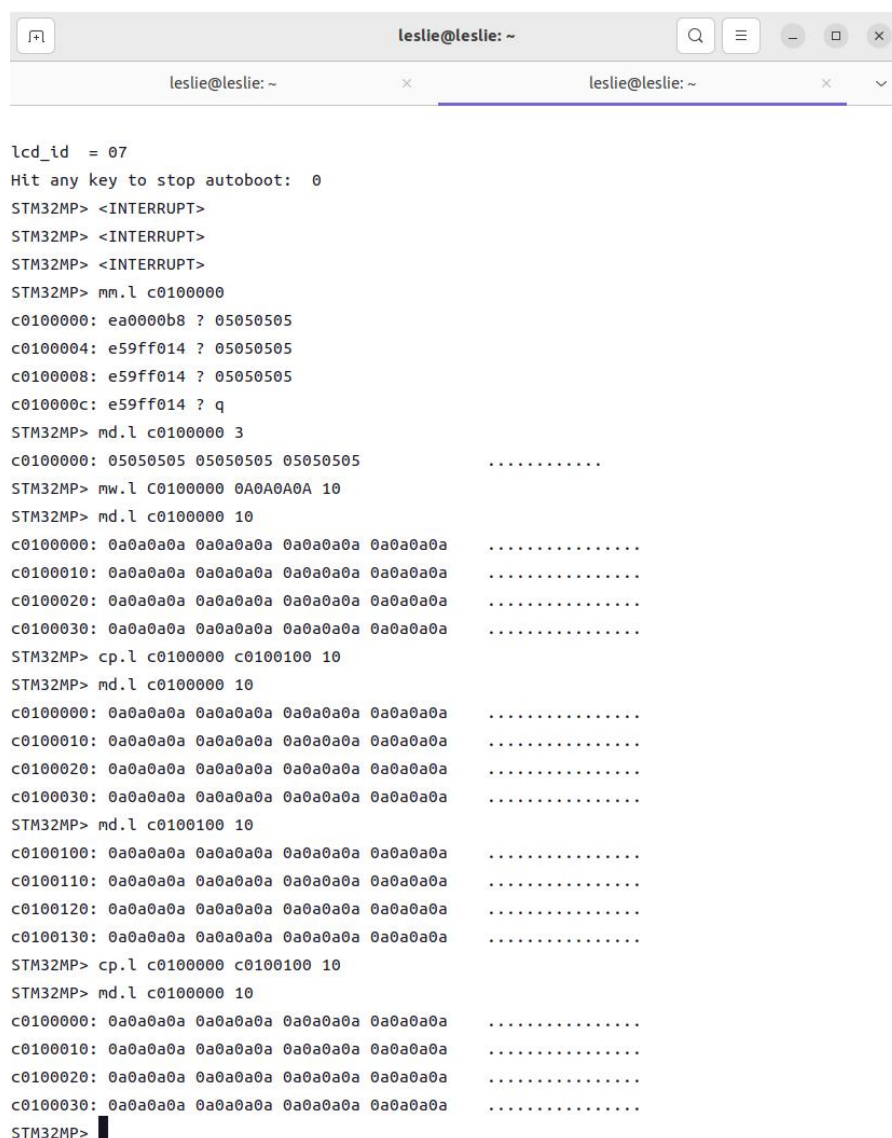
```
leslie@leslie: ~  
leslie@leslie: ~  
Clocks:  
- MPU : 800 MHz  
- MCU : 208.878 MHz  
- AXI : 266.500 MHz  
- PER : 24 MHz  
- DDR : 533 MHz  
WDT: Started with servicing (32s timeout)  
NAND: 0 MiB  
MMC: STM32 SD/MMC: 0, STM32 SD/MMC: 1  
Loading Environment from MMC... OK  
In: serial  
Out: serial  
Err: serial  
invalid MAC address in OTP 00:00:00:00:00:00  
Net:  
Error: ethernet@5800a000 address not set.  
No ethernet found.  
  
lcd_id = 07  
Hit any key to stop autoboot: 0  
STM32MP> <INTERRUPT>  
STM32MP> <INTERRUPT>  
STM32MP> <INTERRUPT>  
STM32MP> mm.l c0100000  
c0100000: ea0000b8 ? 05050505  
c0100004: e59ff014 ? 05050505  
c0100008: e59ff014 ? 05050505  
c010000c: e59ff014 ? q  
STM32MP> md.l c0100000 3  
c0100000: 05050505 05050505 05050505 .....  
STM32MP> mw.l c0100000 0A0A0A0A 10  
STM32MP> md.l c0100000 10  
c0100000: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100010: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100020: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100030: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
STM32MP> █
```

图 3-15 mw 命令填充结果

mw 的作用是用同一个数值批量填充一段区域。与 nm、mm 相比，它不再强调逐项修改，而是强调快速构造统一模式数据。这种命令在调试中非常有用，因为它能迅速把一块复杂区域变成具有明显特征的模式区域，再通过后续命令观察是否发生覆盖或变化。

统一填充模式出现后，说明命令已经对整段目标区域生效。对于大规模测试数据构造、缓冲区初始化和边界验证，mw 比逐项写入要高效得多。

3.4.7 使用 cp 复制内存



```
leslie@leslie: ~  
leslie@leslie: ~  
lcd_id = 07  
Hit any key to stop autoboot: 0  
STM32MP> <INTERRUPT>  
STM32MP> <INTERRUPT>  
STM32MP> <INTERRUPT>  
STM32MP> mm.l c0100000  
c0100000: ea0000b8 ? 05050505  
c0100004: e59ff014 ? 05050505  
c0100008: e59ff014 ? 05050505  
c010000c: e59ff014 ? q  
STM32MP> md.l c0100000 3  
c0100000: 05050505 05050505 05050505 .....  
STM32MP> mw.l c0100000 0A0A0A0A 10  
STM32MP> md.l c0100000 10  
c0100000: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100010: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100020: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100030: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
STM32MP> cp.l c0100000 c0100100 10  
STM32MP> md.l c0100000 10  
c0100000: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100010: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100020: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100030: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
STM32MP> md.l c0100100 10  
c0100100: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100110: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100120: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100130: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
STM32MP> cp.l c0100000 c0100100 10  
STM32MP> md.l c0100000 10  
c0100000: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100010: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100020: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100030: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
STM32MP>
```

图 3-16 cp 命令操作结果

cp 负责把一段数据从一个内存区域搬运到另一个内存区域。镜像装载、缓冲区处理和某些启动过程，在底层都能抽象成这种“数据搬运”行为。虽然实验里的 cp 只是简单演示，但其对应的系统意义非常典型。

命令学习若只停留在“能复制”，就会忽略它真正的重要性。实际上，cp 所训练的是对源地址、目标地址和复制长度三者之间关系的掌握。只要其中任何一个参数理解错误，结果就可能完全偏离预期。

3.4.8 使用 cmp 比较内存

```
leslie@leslie: ~  
leslie@leslie: ~ x leslie@leslie: ~ x v  
Hit any key to stop autoboot: 0  
STM32MP> <INTERRUPT>  
STM32MP> <INTERRUPT>  
STM32MP> <INTERRUPT>  
STM32MP> mm.l c0100000  
c0100000: ea0000b8 ? 05050505  
c0100004: e59ff014 ? 05050505  
c0100008: e59ff014 ? 05050505  
c010000c: e59ff014 ? q  
STM32MP> md.l c0100000 3  
c0100000: 05050505 05050505 05050505 .....  
STM32MP> mw.l c0100000 0A0A0A0A 10  
STM32MP> md.l c0100000 10  
c0100000: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100010: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100020: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100030: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
STM32MP> cp.l c0100000 c0100100 10  
STM32MP> md.l c0100000 10  
c0100000: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100010: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100020: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100030: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
STM32MP> md.l c0100100 10  
c0100100: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100110: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100120: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100130: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
STM32MP> cp.l c0100000 c0100100 10  
STM32MP> md.l c0100000 10  
c0100000: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100010: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100020: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100030: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
STM32MP> cmp.l c0100000 c0100100 10  
Total of 16 word(s) were the same  
STM32MP> █
```

图 3-17 cmp 比较结果一

```
leslie@leslie: ~  
STM32MP> <INTERRUPT>  
STM32MP> mm.l c0100000  
c0100000: ea0000b8 ? 05050505  
c0100004: e59ff014 ? 05050505  
c0100008: e59ff014 ? 05050505  
c010000c: e59ff014 ? q  
STM32MP> md.l c0100000 3  
c0100000: 05050505 05050505 05050505 .....  
STM32MP> mw.l c0100000 0A0A0A0A 10  
STM32MP> md.l c0100000 10  
c0100000: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100010: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100020: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100030: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
STM32MP> cp.l c0100000 c0100100 10  
STM32MP> md.l c0100000 10  
c0100000: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100010: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100020: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100030: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
STM32MP> md.l c0100100 10  
c0100100: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100110: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100120: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100130: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
STM32MP> cp.l c0100000 c0100100 10  
STM32MP> md.l c0100000 10  
c0100000: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100010: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100020: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
c0100030: 0a0a0a0a 0a0a0a0a 0a0a0a0a 0a0a0a0a .....  
STM32MP> cmp.l c0100000 c0100100 10  
Total of 16 word(s) were the same  
STM32MP> cmp.l c0100000 c0100200 10  
word at 0xc0100000 (0xa0a0a0a) != word at 0xc0100200 (0xe24dd048)  
Total of 0 word(s) were the same  
STM32MP> █
```

图 3-18 cmp 比较结果二

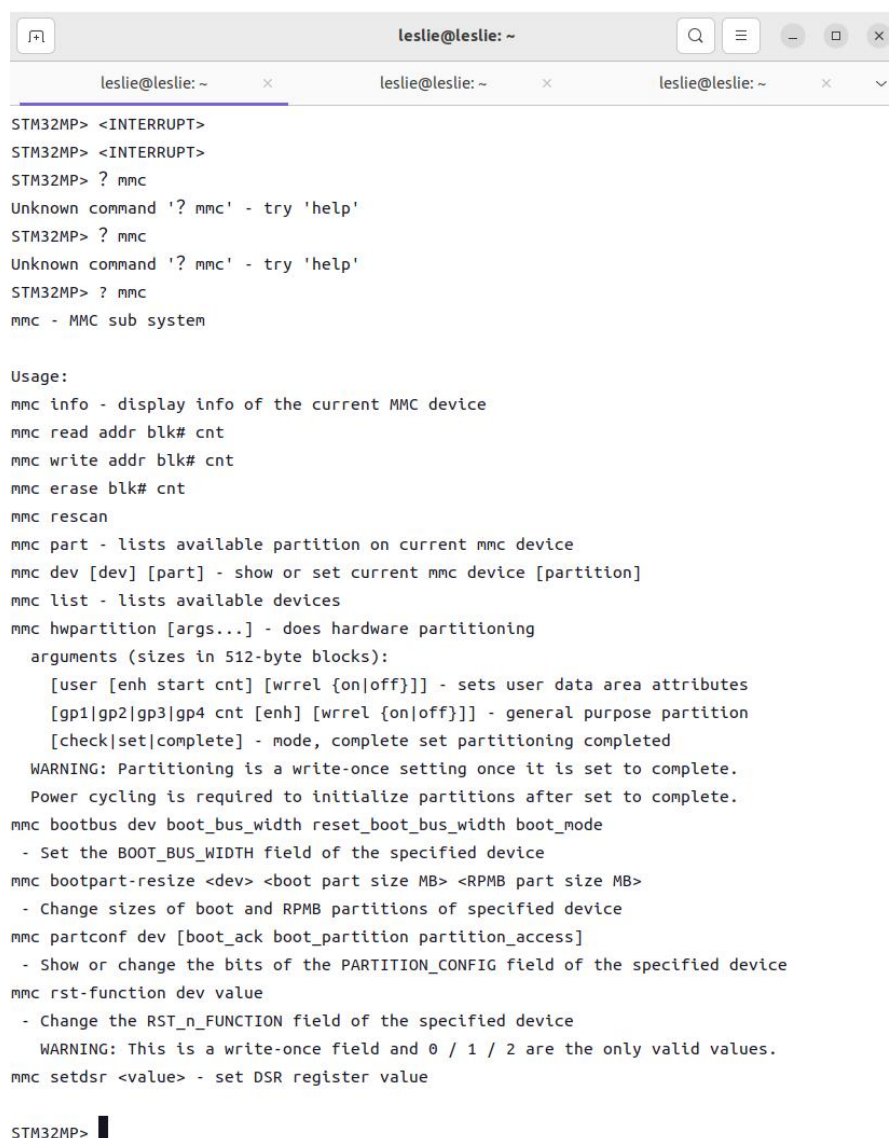
cmp 用于判断两段内存是否一致。实验中保留了相等与不相等两种结果，这使比较命令的行为更加完整。若只保留一种现象，则只能说明命令“有回显”，而不能说明命令“能够对不同输入状态作出不同判断”。

从实际调试角度看，cmp 对于验证 cp 是否真正成功、镜像数据是否完整一致、某块内存是否被无意覆盖都非常有帮助。比较命令的价值不在于输出一句“相等”或“不相等”，而在于它能迅速缩小问题范围。

3.5 任务五：完成 EMMC 与 ext4 文件系统实验

块设备与文件系统实验负责把“外部存储中的镜像文件”与“DRAM 中可供启动的镜像数据”连在一起。若没有这一部分，启动链就会断在“文件存在”与“数据可用”之间。

3.5.1 查看 MMC 命令集合



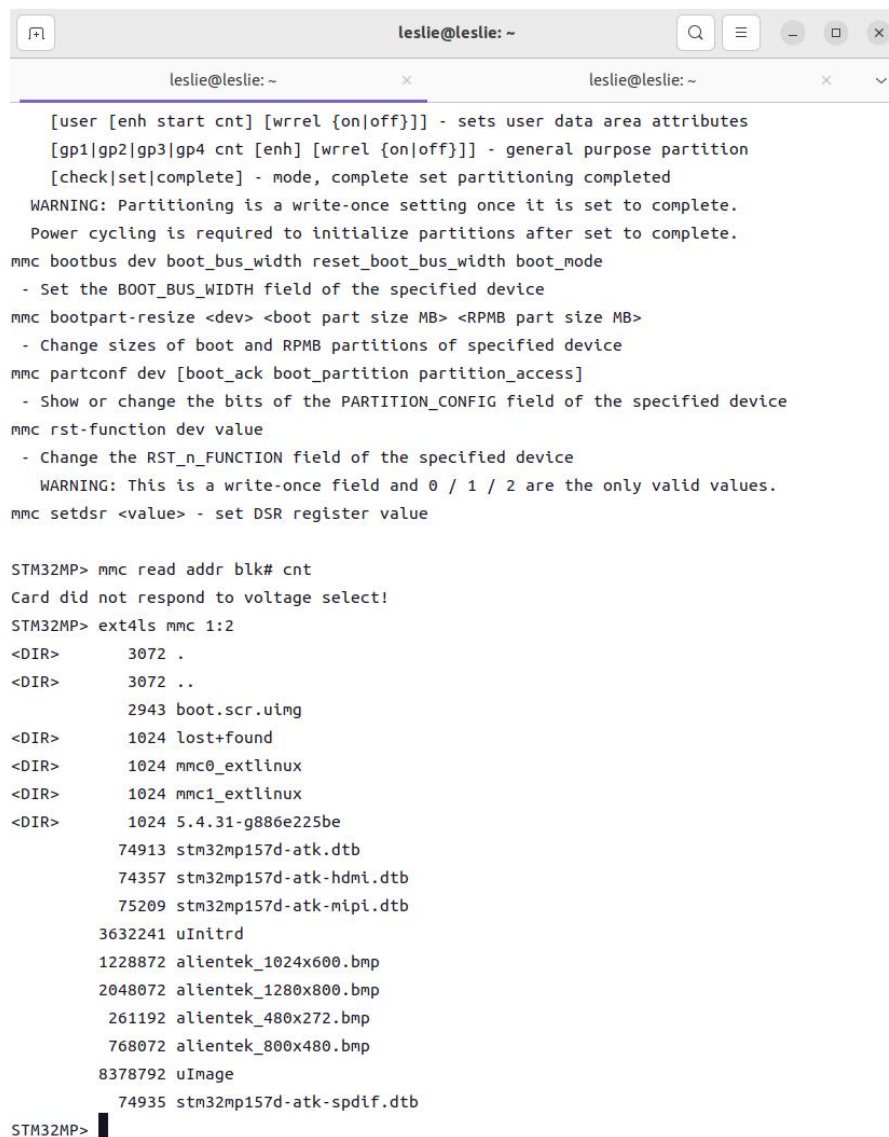
```
leslie@leslie: ~  
leslie@leslie: ~ x leslie@leslie: ~ x leslie@leslie: ~ x  
STM32MP> <INTERRUPT>  
STM32MP> <INTERRUPT>  
STM32MP> ? mmc  
Unknown command '? mmc' - try 'help'  
STM32MP> ? mmc  
Unknown command '? mmc' - try 'help'  
STM32MP> ? mmc  
mmc - MMC sub system  
  
Usage:  
mmc info - display info of the current MMC device  
mmc read addr blk# cnt  
mmc write addr blk# cnt  
mmc erase blk# cnt  
mmc rescan  
mmc part - lists available partition on current mmc device  
mmc dev [dev] [part] - show or set current mmc device [partition]  
mmc list - lists available devices  
mmc hwpartition [args...] - does hardware partitioning  
arguments (sizes in 512-byte blocks):  
[user [enh start cnt] [wrrel {on|off}]] - sets user data area attributes  
[gp1|gp2|gp3|gp4 cnt [enh] [wrrel {on|off}]] - general purpose partition  
[check|set|complete] - mode, complete set partitioning completed  
WARNING: Partitioning is a write-once setting once it is set to complete.  
Power cycling is required to initialize partitions after set to complete.  
mmc bootbus dev boot_bus_width reset_boot_bus_width boot_mode  
- Set the BOOT_BUS_WIDTH field of the specified device  
mmc bootpart-resize <dev> <boot part size MB> <RPMB part size MB>  
- Change sizes of boot and RPMB partitions of specified device  
mmc partconf dev [boot_ack boot_partition partition_access]  
- Show or change the bits of the PARTITION_CONFIG field of the specified device  
mmc rst-function dev value  
- Change the RST_n_FUNCTION field of the specified device  
WARNING: This is a write-once field and 0 / 1 / 2 are the only valid values.  
mmc setdsr <value> - set DSR register value  
  
STM32MP> |
```

图 3-19 mmc 命令与子命令

mmc 并不是单条命令，而是一组围绕块设备展开的操作接口。mmc info、mmc list、mmc dev、mmc part、mmc read 等命令共同完成设备识别、设备切换、分区查看和底层读写。文件系统命令之所以能够继续执行，前提就在于 mmc 阶段已经把设备层信息准备好。

从学习角度看，mmc 小节帮助区分“设备”“分区”“文件系统”“文件”这四个层次。设备是最底层对象，文件是最高层对象。若这几个层次混淆，后续排错就容易出现方向错位。

3.5.2 查询 ext4 分区中的文件



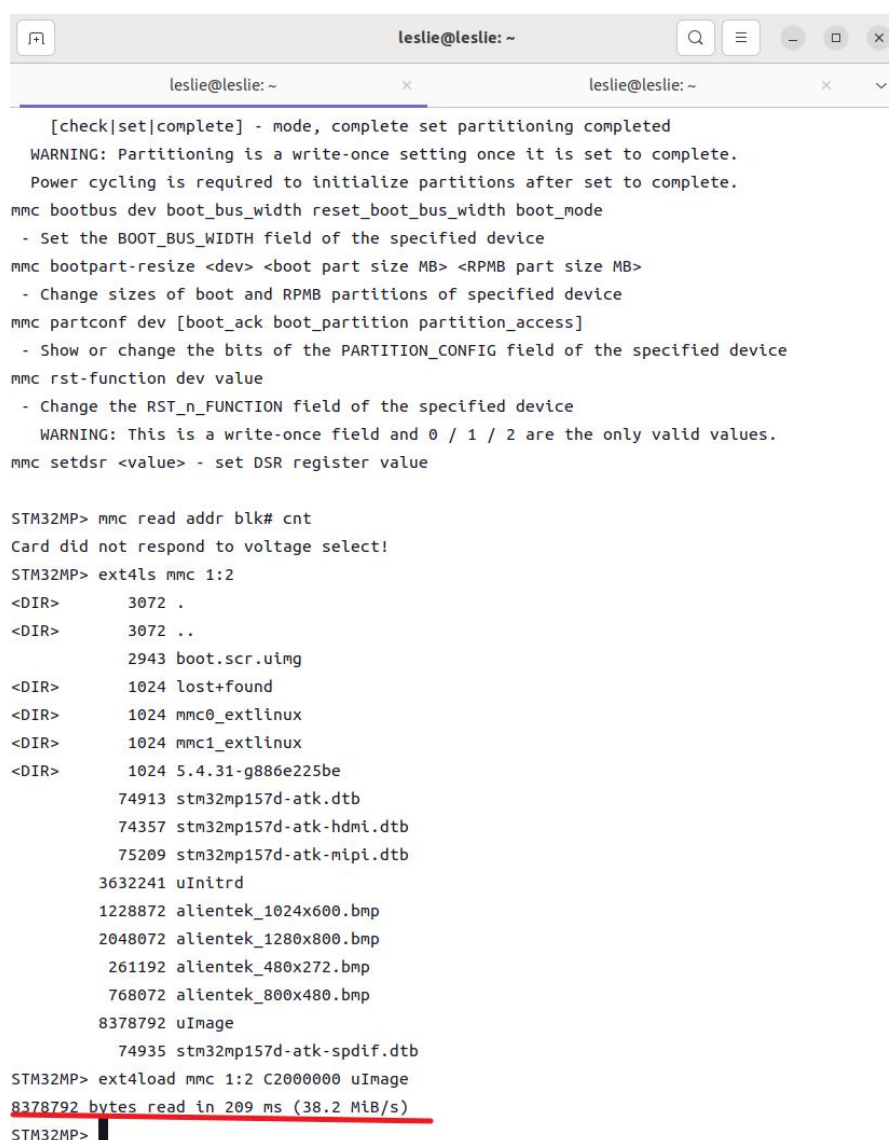
```
leslie@leslie: ~  
[user [enh start cnt] [wrrel {on|off}]] - sets user data area attributes  
[gp1|gp2|gp3|gp4 cnt [enh] [wrrel {on|off}]] - general purpose partition  
[check|set|complete] - mode, complete set partitioning completed  
WARNING: Partitioning is a write-once setting once it is set to complete.  
Power cycling is required to initialize partitions after set to complete.  
mmc bootbus dev boot_bus_width reset_boot_bus_width boot_mode  
- Set the BOOT_BUS_WIDTH field of the specified device  
mmc bootpart-resize <dev> <boot part size MB> <RPMB part size MB>  
- Change sizes of boot and RPMB partitions of specified device  
mmc partconf dev [boot_ack boot_partition partition_access]  
- Show or change the bits of the PARTITION_CONFIG field of the specified device  
mmc rst-function dev value  
- Change the RST_n_FUNCTION field of the specified device  
WARNING: This is a write-once field and 0 / 1 / 2 are the only valid values.  
mmc setdsr <value> - set DSR register value  
  
STM32MP> mmc read addr blk# cnt  
Card did not respond to voltage select!  
STM32MP> ext4ls mmc 1:2  
<DIR>      3072 .  
<DIR>      3072 ..  
          2943 boot.scr.uimg  
<DIR>      1024 lost+found  
<DIR>      1024 mmc0_extlinux  
<DIR>      1024 mmc1_extlinux  
<DIR>      1024 5.4.31-g886e225be  
          74913 stm32mp157d-atk.dtb  
          74357 stm32mp157d-atk-hdmi.dtb  
          75209 stm32mp157d-atk-mipi.dtb  
          3632241 uInitrd  
          1228872 alientek_1024x600.bmp  
          2048072 alientek_1280x800.bmp  
          261192 alientek_480x272.bmp  
          768072 alientek_800x480.bmp  
          8378792 uImage  
          74935 stm32mp157d-atk-spdif.dtb  
STM32MP>
```

图 3-20 ext4ls 查询结果

ext4ls 负责列出 ext4 分区中的文件信息。从实验结果中能够看到 uImage 和设备树文件已经存在于目标分区中，这说明后续装载操作具备了前提条件。启动命令不能脱离镜像文件独立存在，镜像文件是否存在，永远应先于启动命令被确认。

在调试中，若 ext4ls 查不到文件，首先应怀疑设备号、分区号、路径或文件名，而不应直接把问题归结为 bootm 或 bootcmd。这也是课程实验通过查询命令和启动命令分阶段安排的原因。

3.5.3 读取 uImage 到 DRAM



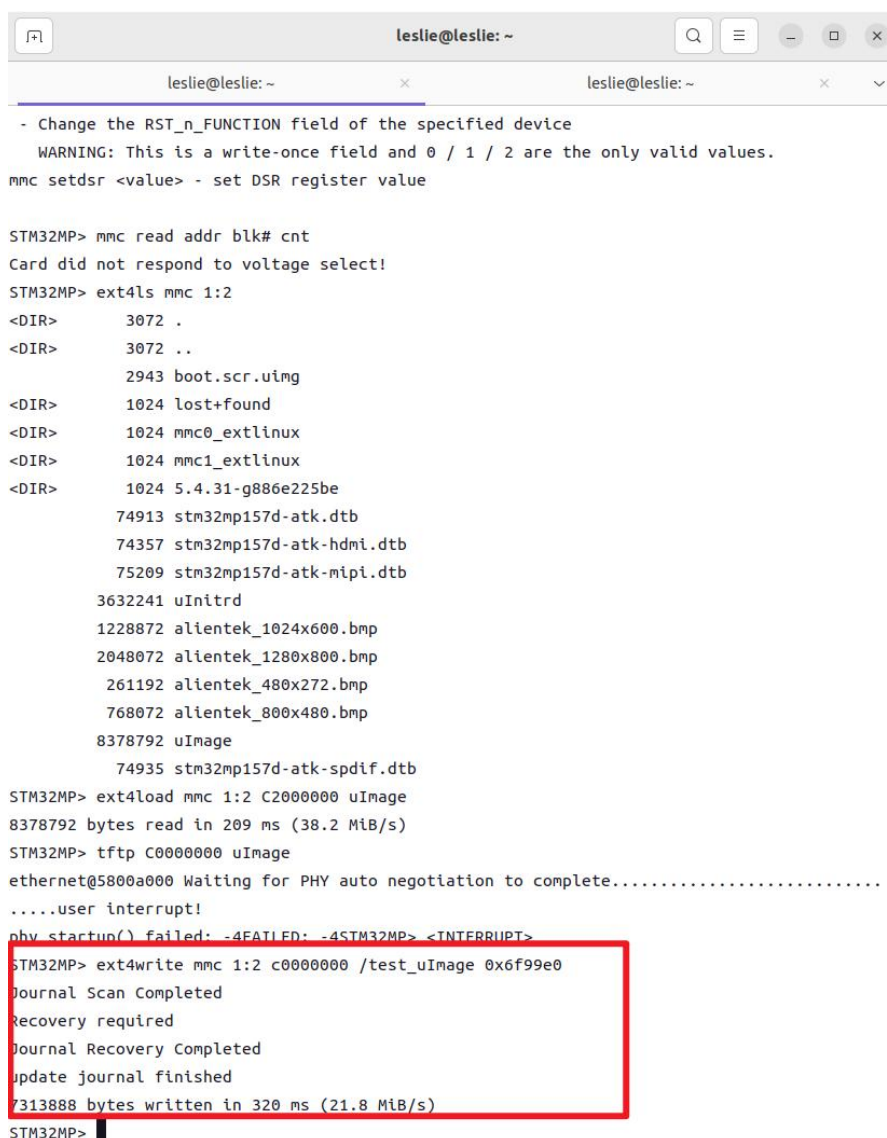
```
leslie@leslie: ~  
[check|set|complete] - mode, complete set partitioning completed  
WARNING: Partitioning is a write-once setting once it is set to complete.  
Power cycling is required to initialize partitions after set to complete.  
mmc bootbus dev boot_bus_width reset_boot_bus_width boot_mode  
- Set the BOOT_BUS_WIDTH field of the specified device  
mmc bootpart-resize <dev> <boot part size MB> <RPMB part size MB>  
- Change sizes of boot and RPMB partitions of specified device  
mmc partconf dev [boot_ack boot_partition partition_access]  
- Show or change the bits of the PARTITION_CONFIG field of the specified device  
mmc rst-function dev value  
- Change the RST_n_FUNCTION field of the specified device  
WARNING: This is a write-once field and 0 / 1 / 2 are the only valid values.  
mmc setdsr <value> - set DSR register value  
  
STM32MP> mmc read addr blk# cnt  
Card did not respond to voltage select!  
STM32MP> ext4ls mmc 1:2  
<DIR>      3072 .  
<DIR>      3072 ..  
      2943 boot.scr.uimg  
<DIR>      1024 lost+found  
<DIR>      1024 mmc0_extlinux  
<DIR>      1024 mmc1_extlinux  
<DIR>      1024 5.4.31-g886e225be  
      74913 stm32mp157d-atk.dtb  
      74357 stm32mp157d-atk-hdmi.dtb  
      75209 stm32mp157d-atk-mipi.dtb  
      3632241 uInitrd  
      1228872 alientek_1024x600.bmp  
      2048072 alientek_1280x800.bmp  
      261192 alientek_480x272.bmp  
      768072 alientek_800x480.bmp  
      8378792 uImage  
      74935 stm32mp157d-atk-spdif.dtb  
STM32MP> ext4load mmc 1:2 C2000000 uImage  
8378792 bytes read in 209 ms (38.2 MiB/s)  
STM32MP>
```

图 3-21 ext4load 读取过程

ext4load mmc 1:2 C2000000 uImage 的含义，是从 mmc 1:2 对应的 ext4 分区中读取 uImage 文件，并把它装载到 C2000000 地址。这个动作真正把“文件系统里的文件”转换成了“内存中的镜像数据”。也正是在这一刻，镜像才从存储资源转化为启动资源。

这条命令最关键的参数有三个：设备与分区、目标地址、文件名。任何一项有误，都会直接导致装载失败或后续启动异常。因此，ext4load 的成功并不意味着文件可读，也意味着地址规划、设备选择和文件定位三者已经匹配。

3.5.4 把数据写回 ext4 分区



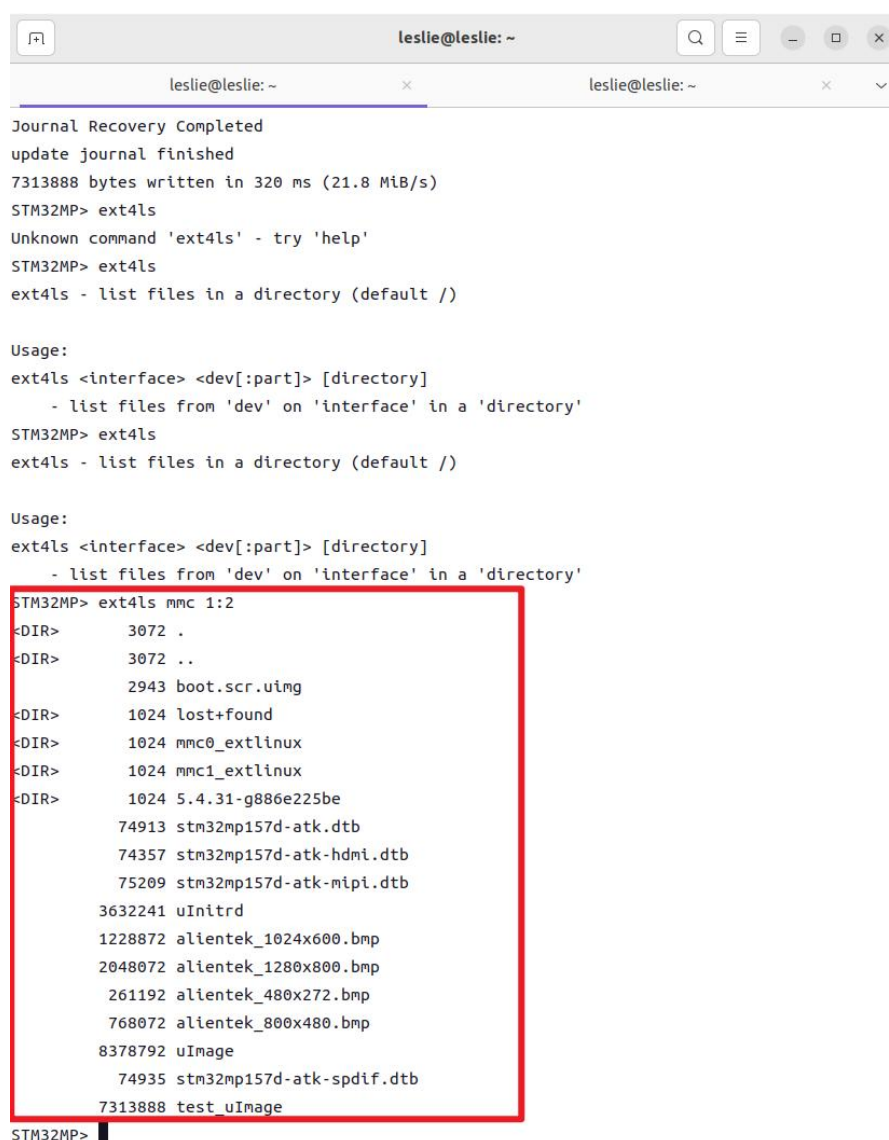
```
leslie@leslie: ~  
- Change the RST_n_FUNCTION field of the specified device  
WARNING: This is a write-once field and 0 / 1 / 2 are the only valid values.  
mmc setdsr <value> - set DSR register value  
  
STM32MP> mmc read addr blk# cnt  
Card did not respond to voltage select!  
STM32MP> ext4ls mmc 1:2  
<DIR>      3072 .  
<DIR>      3072 ..  
          2943 boot.scr.uimg  
<DIR>      1024 lost+found  
<DIR>      1024 mmc0_extlinux  
<DIR>      1024 mmc1_extlinux  
<DIR>      1024 5.4.31-g886e225be  
          74913 stm32mp157d-atk.dtb  
          74357 stm32mp157d-atk-hdmi.dtb  
          75209 stm32mp157d-atk-mipi.dtb  
          3632241 uInitrd  
          1228872 alientek_1024x600.bmp  
          2048072 alientek_1280x800.bmp  
          261192 alientek_480x272.bmp  
          768072 alientek_800x480.bmp  
          8378792 uImage  
          74935 stm32mp157d-atk-spdif.dtb  
STM32MP> ext4load mmc 1:2 C2000000 uImage  
8378792 bytes read in 209 ms (38.2 MiB/s)  
STM32MP> tftp C0000000 uImage  
ethernet@5800a000 Waiting for PHY auto negotiation to complete.....  
....user interrupt!  
phy_startup() failed: -4EATIED: -4STM32MP> <INTERRUPT>  
STM32MP> ext4write mmc 1:2 C0000000 /test_uImage 0x6f99e0  
Journal Scan Completed  
Recovery required  
Journal Recovery Completed  
update journal finished  
7313888 bytes written in 320 ms (21.8 MiB/s)  
STM32MP>
```

图 3-22 ext4write 写入过程

ext4write 所体现的是从 DRAM 到文件系统的反向路径。若 ext4load 证明 U-Boot 能“读文件进内存”，那么 ext4write 证明 U-Boot 也能“把内存内容写成文件”。读写双向能力具备之后，U-Boot 对文件系统的作用就不再局限于启动前读取，而扩展到维护与更新。

在实际场景中，这类能力非常有价值。若 Linux 尚未启动而系统文件又需要替换，就可以在 U-Boot 阶段直接完成写入，避免必须进入上层系统之后才能维护。

3.5.5 再次查询新文件



```
leslie@leslie: ~  
leslie@leslie: ~  
Journal Recovery Completed  
update journal finished  
7313888 bytes written in 320 ms (21.8 MiB/s)  
STM32MP> ext4ls  
Unknown command 'ext4ls' - try 'help'  
STM32MP> ext4ls  
ext4ls - list files in a directory (default /)  
  
Usage:  
ext4ls <interface> <dev[:part]> [directory]  
- list files from 'dev' on 'interface' in a 'directory'  
STM32MP> ext4ls  
ext4ls - list files in a directory (default /)  
  
Usage:  
ext4ls <interface> <dev[:part]> [directory]  
- list files from 'dev' on 'interface' in a 'directory'  
STM32MP> ext4ls mmc 1:2  
<DIR>      3072 .  
<DIR>      3072 ..  
          2943 boot.scr.uimg  
<DIR>      1024 lost+found  
<DIR>      1024 mmc0_extlinux  
<DIR>      1024 mmc1_extlinux  
<DIR>      1024 5.4.31-g886e225be  
          74913 stm32mp157d-atk.dtb  
          74357 stm32mp157d-atk-hdmi.dtb  
          75209 stm32mp157d-atk-mipi.dtb  
          3632241 uInitrd  
          1228872 alientek_1024x600.bmp  
          2048072 alientek_1280x800.bmp  
          261192 alientek_480x272.bmp  
          768072 alientek_800x480.bmp  
          8378792 uImage  
          74935 stm32mp157d-atk-spdif.dtb  
          7313888 test_uImage  
STM32MP>
```

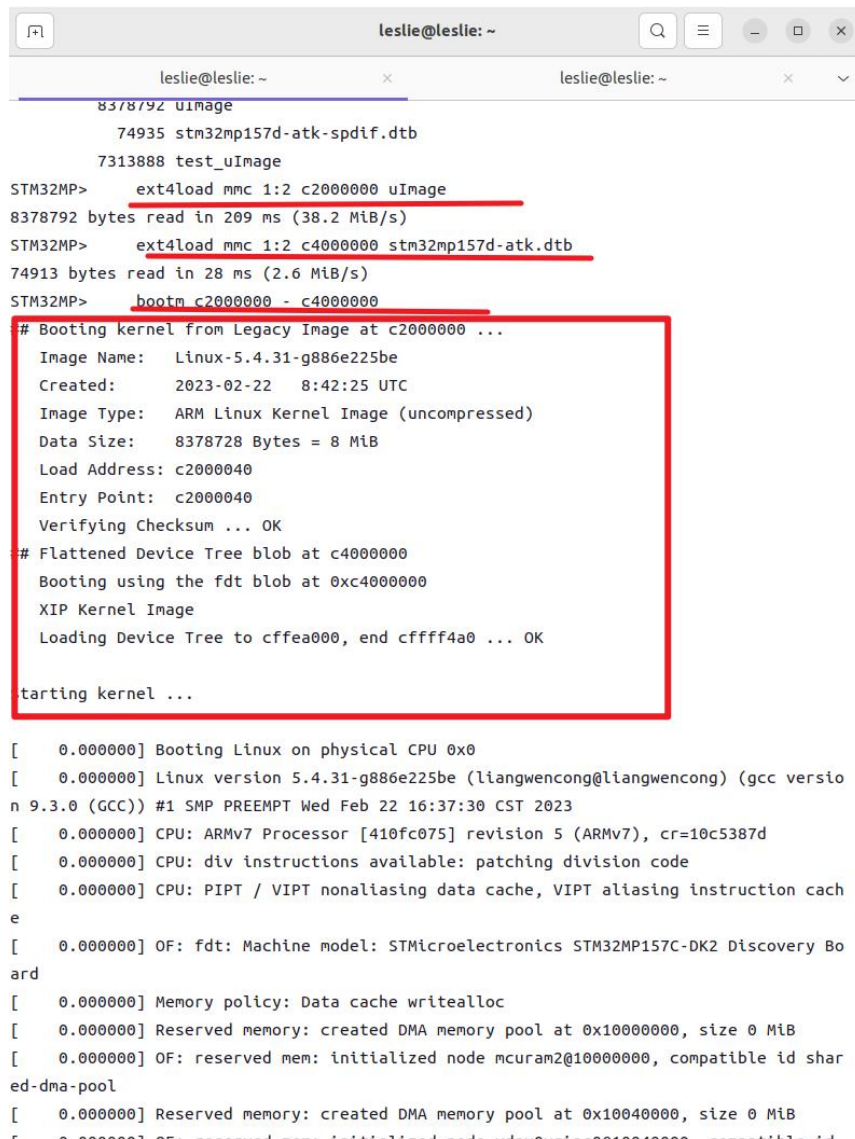
图 3-23 新文件查询结果

写入之后再次使用 `ext4ls` 查看分区内容，已经能够看到新生成的 `test_uImage` 文件。这里的逻辑闭环与前面的内存和环境变量实验完全一致：写入并不自动等于写入成功，只有再次观察文件系统状态，结论才成立。

这一步使 `ext4` 实验链形成完整闭环：先查文件，再读文件，再写文件，再查新文件。这样整理之后，U-Boot 在 `ext4` 文件系统层面的基本能力已经被连续验证。

3.6 任务六：完成 Linux 启动验证

3.6.1 从 EMMC 手动启动 Linux



```
leslie@leslie: ~
8378792 uImage
74935 stm32mp157d-atk-spdif.dtb
7313888 test_uImage
STM32MP> ext4load mmc 1:2 c2000000 uImage
8378792 bytes read in 209 ms (38.2 MiB/s)
STM32MP> ext4load mmc 1:2 c4000000 stm32mp157d-atk.dtb
74913 bytes read in 28 ms (2.6 MiB/s)
STM32MP> bootm c2000000 - c4000000
# Booting kernel from Legacy Image at c2000000 ...
Image Name: Linux-5.4.31-g886e225be
Created: 2023-02-22 8:42:25 UTC
Image Type: ARM Linux Kernel Image (uncompressed)
Data Size: 8378728 Bytes = 8 MiB
Load Address: c2000040
Entry Point: c2000040
Verifying Checksum ... OK
# Flattened Device Tree blob at c4000000
Booting using the fdt blob at 0xc4000000
XIP Kernel Image
Loading Device Tree to cffea000, end cffff4a0 ... OK
tarting kernel ...

[ 0.000000] Booting Linux on physical CPU 0x0
[ 0.000000] Linux version 5.4.31-g886e225be (liangwencong@liangwencong) (gcc versio
n 9.3.0 (GCC)) #1 SMP PREEMPT Wed Feb 22 16:37:30 CST 2023
[ 0.000000] CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c5387d
[ 0.000000] CPU: div instructions available: patching division code
[ 0.000000] CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing instruction cach
e
[ 0.000000] OF: fdt: Machine model: STMicroelectronics STM32MP157C-DK2 Discovery Bo
ard
[ 0.000000] Memory policy: Data cache writealloc
[ 0.000000] Reserved memory: created DMA memory pool at 0x10000000, size 0 MiB
[ 0.000000] OF: reserved mem: initialized node mcuram2@10000000, compatible id shar
ed-dma-pool
[ 0.000000] Reserved memory: created DMA memory pool at 0x10040000, size 0 MiB
[ 0.000000] OF: reserved mem: initialized node mcuram2@10040000, compatible id shar
ed-dma-pool
```

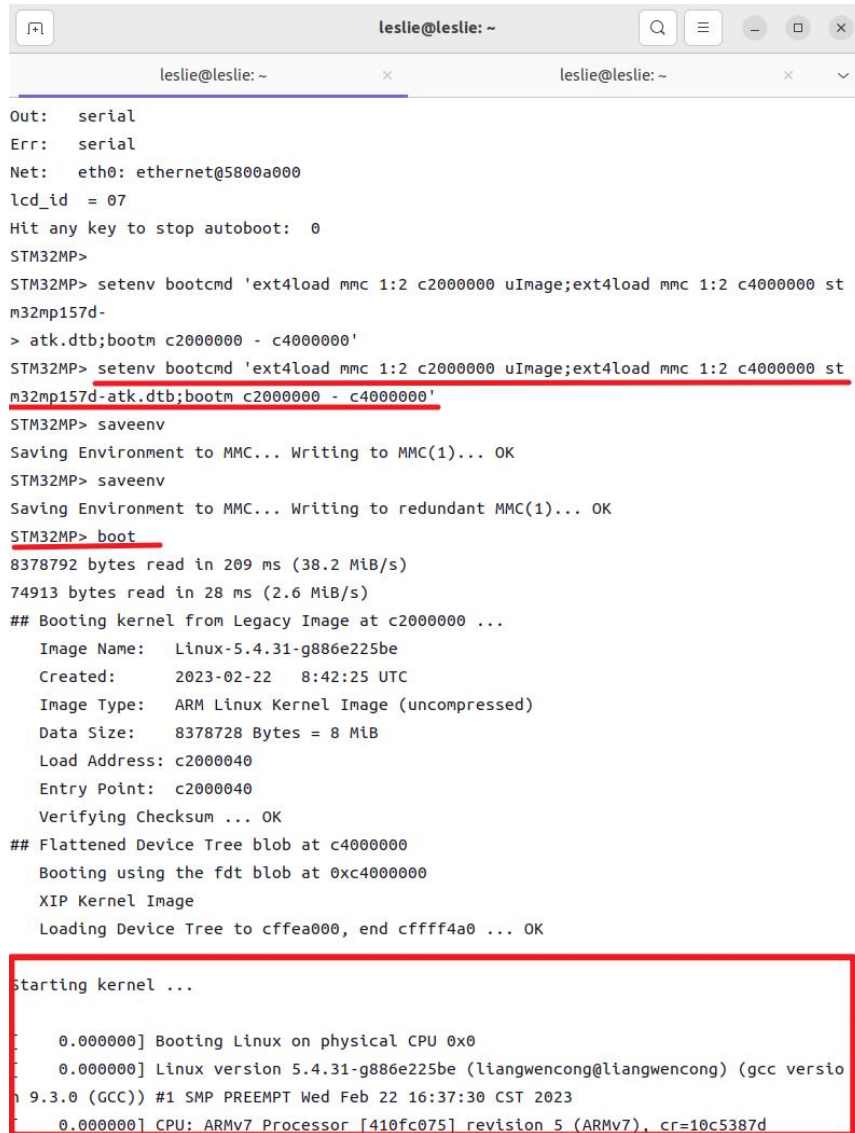
图 3-24 从 EMMC 启动 Linux

从实验记录可见，系统先通过 `ext4load` 把 `uImage` 和 `stm32mp157d-atk.dtb` 读入 DRAM，然后执行 `bootm c2000000 - c4000000`。只要 Linux 内核日志开始输出，就说明 U-Boot 已经把镜像和设备树正确交给了内核。

这一过程是整套实验中最关键的节点。因为此前所有查询、变量、内存和文件系统操作，归根到底都在为“能否完成内核引导”服务。若没有这一启动环节，前面的命令会显得分散；而一旦启动成功，前面所有内容就自然汇合成一条完整链路。

需要特别区分的是：内核日志出现，只能说明 U-Boot 启动阶段成功，不等于 Linux 一定已经完整进入用户空间。若后续出现根文件系统挂载问题，那么故障分析重点应转向 bootargs 与根文件系统，而不是重新怀疑 bootm 本身。

3.6.2 设置默认启动脚本



```
leslie@leslie: ~
leslie@leslie: ~
leslie@leslie: ~
Out: serial
Err: serial
Net: eth0: ethernet@5800a000
lcd_id = 07
Hit any key to stop autoboot: 0
STM32MP>
STM32MP> setenv bootcmd 'ext4load mmc 1:2 c2000000 uImage;ext4load mmc 1:2 c4000000 stm32mp157d-atk.dtb;bootm c2000000 - c4000000'
STM32MP> saveenv
Saving Environment to MMC... Writing to MMC(1)... OK
STM32MP> saveenv
Saving Environment to MMC... Writing to redundant MMC(1)... OK
STM32MP> boot
8378792 bytes read in 209 ms (38.2 MiB/s)
74913 bytes read in 28 ms (2.6 MiB/s)
## Booting kernel from Legacy Image at c2000000 ...
   Image Name:   Linux-5.4.31-g886e225be
   Created:      2023-02-22   8:42:25 UTC
   Image Type:   ARM Linux Kernel Image (uncompressed)
   Data Size:    8378728 Bytes = 8 MiB
   Load Address: c2000040
   Entry Point:  c2000040
   Verifying Checksum ... OK
## Flattened Device Tree blob at c4000000
   Booting using the fdt blob at 0xc4000000
   XIP Kernel Image
   Loading Device Tree to cffea000, end cffff4a0 ... OK

starting kernel ...

0.000000] Booting Linux on physical CPU 0x0
0.000000] Linux version 5.4.31-g886e225be (liangwencong@liangwencong) (gcc version 9.3.0 (GCC)) #1 SMP PREEMPT Wed Feb 22 16:37:30 CST 2023
0.000000] CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c5387d
```

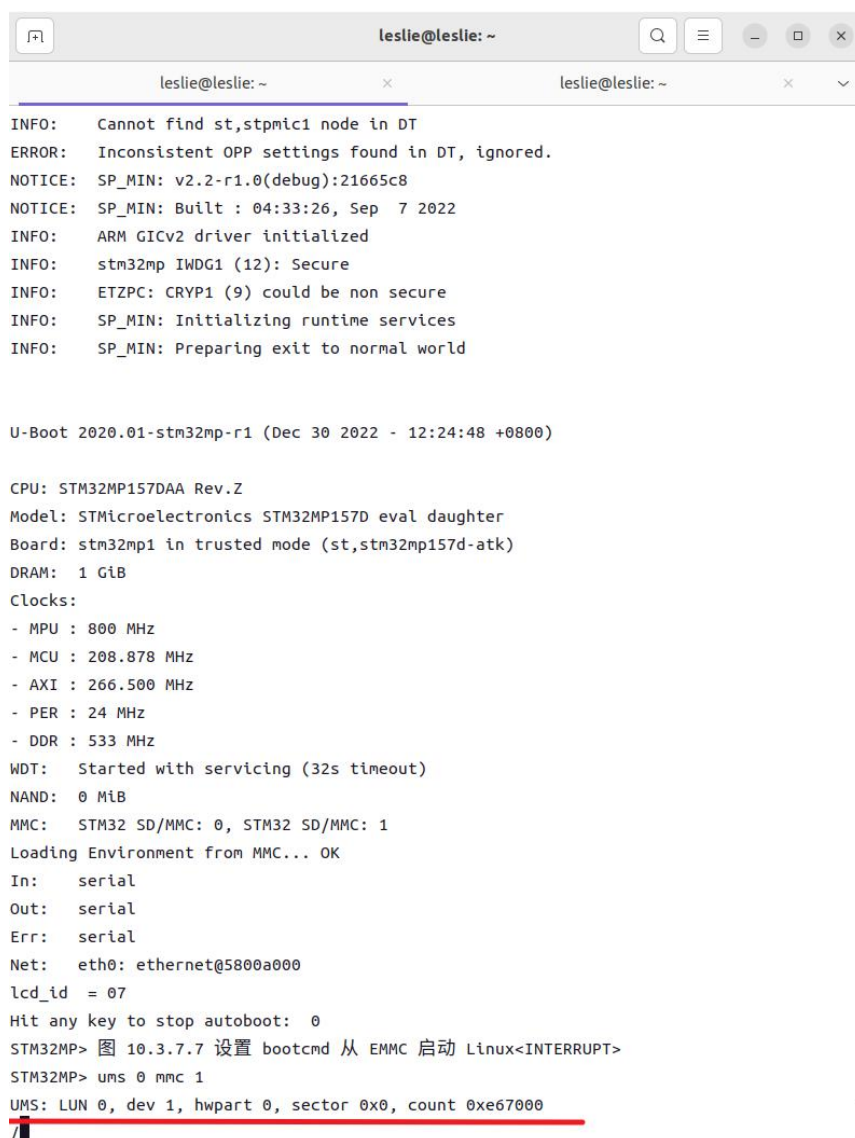
图 3-25 设置 bootcmd

bootcmd 的作用是把手工输入的多条启动命令固化为默认启动脚本。这样一来，系统每次上电后只要倒计时结束，就会自动执行这组命令，不需要再次人工逐条输入。对嵌入式设备而言，这是一种更接近实际部署状态的工作方式。

把 bootcmd 设置成功，意味着对 U-Boot 的控制能力从“会手动启动一次”提升到了“会配置系统默认如何启动”。这一步与环境变量小节有直接联系，因为 bootcmd 本身就是一个环境变量，而默认启动行为又完全由它决定。

3.7 任务七：完成扩展命令验证

3.7.1 验证 ums



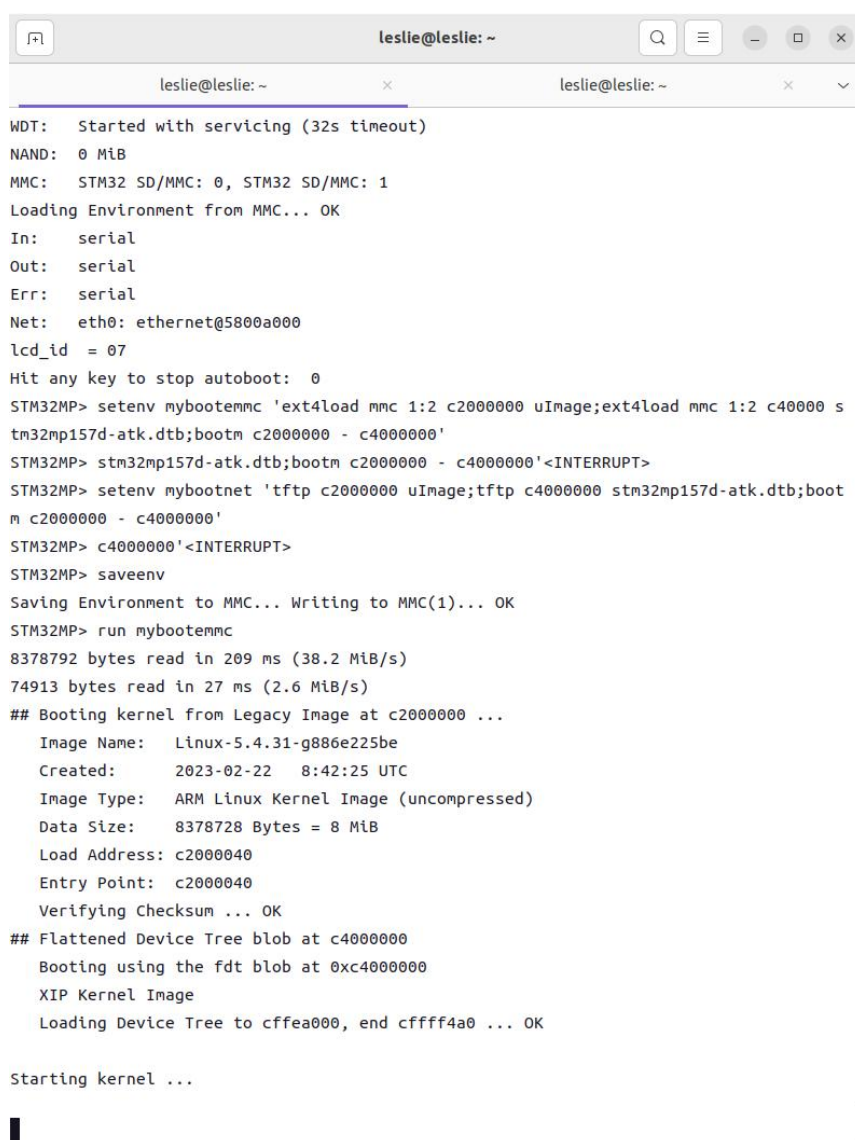
```
leslie@leslie: ~  
leslie@leslie: ~  
INFO: Cannot find st,stm32mp1 node in DT  
ERROR: Inconsistent OPP settings found in DT, ignored.  
NOTICE: SP_MIN: v2.2-r1.0(debug):21665c8  
NOTICE: SP_MIN: Built : 04:33:26, Sep 7 2022  
INFO: ARM GICv2 driver initialized  
INFO: stm32mp IWDG1 (12): Secure  
INFO: ETZPC: CRY1 (9) could be non secure  
INFO: SP_MIN: Initializing runtime services  
INFO: SP_MIN: Preparing exit to normal world  
  
U-Boot 2020.01-stm32mp-r1 (Dec 30 2022 - 12:24:48 +0800)  
  
CPU: STM32MP157DAA Rev.Z  
Model: STMicroelectronics STM32MP157D eval daughter  
Board: stm32mp1 in trusted mode (st,stm32mp157d-atk)  
DRAM: 1 GiB  
Clocks:  
- MPU : 800 MHz  
- MCU : 208.878 MHz  
- AXI : 266.500 MHz  
- PER : 24 MHz  
- DDR : 533 MHz  
WDT: Started with servicing (32s timeout)  
NAND: 0 MiB  
MMC: STM32 SD/MMC: 0, STM32 SD/MMC: 1  
Loading Environment from MMC... OK  
In: serial  
Out: serial  
Err: serial  
Net: eth0: ethernet@5800a000  
lcd_id = 07  
Hit any key to stop autoboot: 0  
STM32MP> 图 10.3.7.7 设置 bootcmd 从 EMMC 启动 Linux<INTERRUPT>  
STM32MP> ums 0 mmc 1  
UMS: LUN 0, dev 1, hwpart 0, sector 0x0, count 0xe67000
```

图 3-26 ums 命令运行结果

ums 0 mmc 1 的作用是把开发板上的 EMMC 虚拟成 USB 大容量存储设备，让主机像访问 U 盘一样访问它。该命令表明，U-Boot 在启动之外还具备一定的维护能力。系统即使没有进入 Linux，仍可以通过这种方式暴露存储内容。

需要注意的是，若主机环境为 Windows，面对 ext4 分区很可能会弹出格式化提示。这个提示不能接受，否则可能直接破坏开发板中的系统分区结构。也正因为存在这种风险，ums 的实验结果不仅要看“能否挂载”，也要结合主机端文件系统支持情况分析。

3.7.2 验证 run



```
leslie@leslie: ~
leslie@leslie: ~
leslie@leslie: ~

WDT:  Started with servicing (32s timeout)
NAND:  0 MiB
MMC:   STM32 SD/MMC: 0, STM32 SD/MMC: 1
Loading Environment from MMC... OK
In:    serial
Out:   serial
Err:   serial
Net:   eth0: ethernet@5800a000
lcd_id = 07
Hit any key to stop autoboot:  0
STM32MP> setenv mybootmmc 'ext4load mmc 1:2 c2000000 uImage;ext4load mmc 1:2 c40000 s
tm32mp157d-atk.dtb;bootm c2000000 - c4000000'
STM32MP> stm32mp157d-atk.dtb;bootm c2000000 - c4000000'<INTERUPT>
STM32MP> setenv mybootnet 'tftp c2000000 uImage;tftp c4000000 stm32mp157d-atk.dtb;boot
m c2000000 - c4000000'
STM32MP> c4000000'<INTERUPT>
STM32MP> saveenv
Saving Environment to MMC... Writing to MMC(1)... OK
STM32MP> run mybootmmc
8378792 bytes read in 209 ms (38.2 MiB/s)
74913 bytes read in 27 ms (2.6 MiB/s)
## Booting kernel from Legacy Image at c2000000 ...
   Image Name:   Linux-5.4.31-g886e225be
   Created:      2023-02-22   8:42:25 UTC
   Image Type:   ARM Linux Kernel Image (uncompressed)
   Data Size:    8378728 Bytes = 8 MiB
   Load Address: c2000040
   Entry Point:  c2000040
   Verifying Checksum ... OK
## Flattened Device Tree blob at c4000000
   Booting using the fdt blob at 0xc4000000
   XIP Kernel Image
   Loading Device Tree to cffea000, end cffff4a0 ... OK

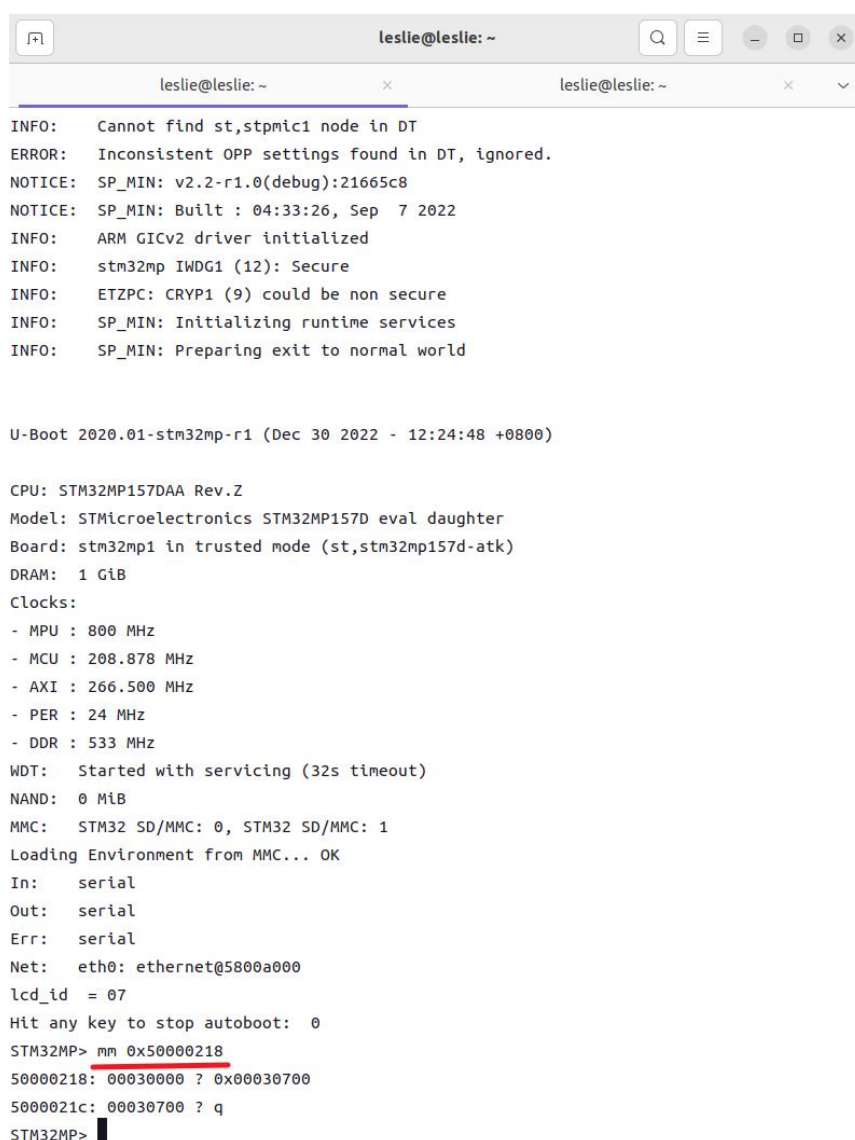
Starting kernel ...
```

图 3-27 run 命令运行结果

run 负责执行环境变量中预先写好的命令序列。它的价值在于把多条命令封装为可复用模块，而不是每次都重新输入。这种机制使不同启动策略之间的切换更加灵活。例如可以把 EMMC 启动、网络启动或调试脚本分别保存为不同变量，再通过 **run** 切换。

从命令体系角度看，**run** 进一步证明环境变量不仅能存参数，也能存脚本逻辑。环境变量与启动脚本之间的关系，通过这一命令体现得最为直接。

3.7.3 使能 ETHMAC 时钟

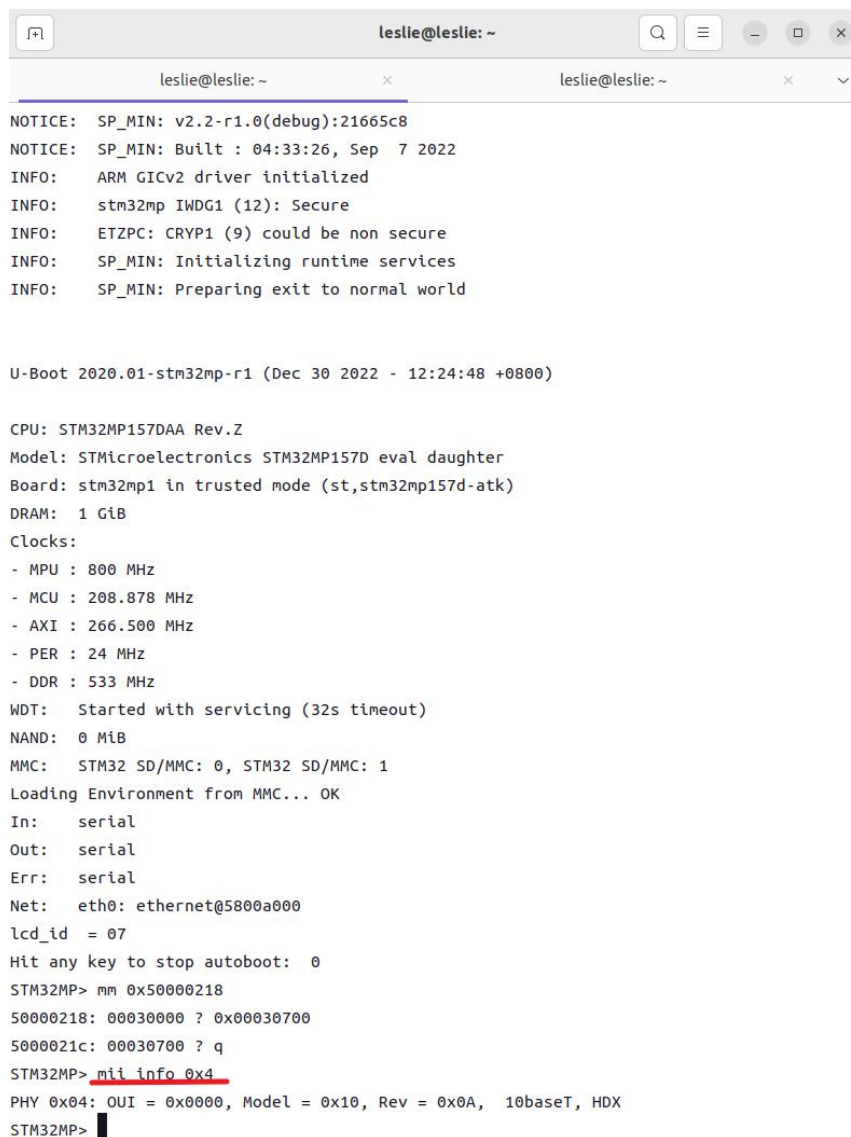


```
leslie@leslie: ~  
leslie@leslie: ~  
INFO: Cannot find st,stm32mp1 node in DT  
ERROR: Inconsistent OPP settings found in DT, ignored.  
NOTICE: SP_MIN: v2.2-r1.0(debug):21665c8  
NOTICE: SP_MIN: Built : 04:33:26, Sep 7 2022  
INFO: ARM GICv2 driver initialized  
INFO: stm32mp IWDG1 (12): Secure  
INFO: ETZPC: CRY1 (9) could be non secure  
INFO: SP_MIN: Initializing runtime services  
INFO: SP_MIN: Preparing exit to normal world  
  
U-Boot 2020.01-stm32mp-r1 (Dec 30 2022 - 12:24:48 +0800)  
  
CPU: STM32MP157DAA Rev.Z  
Model: STMicroelectronics STM32MP157D eval daughter  
Board: stm32mp1 in trusted mode (st,stm32mp157d-atk)  
DRAM: 1 GiB  
Clocks:  
- MPU : 800 MHz  
- MCU : 208.878 MHz  
- AXI : 266.500 MHz  
- PER : 24 MHz  
- DDR : 533 MHz  
WDT: Started with servicing (32s timeout)  
NAND: 0 MiB  
MMC: STM32 SD/MMC: 0, STM32 SD/MMC: 1  
Loading Environment from MMC... OK  
In: serial  
Out: serial  
Err: serial  
Net: eth0: ethernet@5800a000  
lcd_id = 07  
Hit any key to stop autoboot: 0  
STM32MP> mm 0x50000218  
50000218: 00030000 ? 0x00030700  
5000021c: 00030700 ? q  
STM32MP>
```

图 3-28 使能 ETHMAC 时钟

在执行 `mii` 相关命令之前，需要先确保网络控制器相关时钟已经满足工作条件。若底层时钟关闭，即使命令语法正确，也可能因为硬件状态不满足而失败。该实验现象说明，U-Boot 中某些命令是否有效，并不只取决于命令本身，还取决于硬件前置条件是否已满足。

3.7.4 验证 mii info



```
leslie@leslie: ~  
leslie@leslie: ~  
NOTICE: SP_MIN: v2.2-r1.0(debug):21665c8  
NOTICE: SP_MIN: Built : 04:33:26, Sep 7 2022  
INFO: ARM GICv2 driver initialized  
INFO: stm32mp IWDG1 (12): Secure  
INFO: ETZPC: CRY1 (9) could be non secure  
INFO: SP_MIN: Initializing runtime services  
INFO: SP_MIN: Preparing exit to normal world  
  
U-Boot 2020.01-stm32mp-r1 (Dec 30 2022 - 12:24:48 +0800)  
  
CPU: STM32MP157DAA Rev.Z  
Model: STMicroelectronics STM32MP157D eval daughter  
Board: stm32mp1 in trusted mode (st,stm32mp157d-atk)  
DRAM: 1 GiB  
Clocks:  
- MPU : 800 MHz  
- MCU : 208.878 MHz  
- AXI : 266.500 MHz  
- PER : 24 MHz  
- DDR : 533 MHz  
WDT: Started with servicing (32s timeout)  
NAND: 0 MiB  
MMC: STM32 SD/MMC: 0, STM32 SD/MMC: 1  
Loading Environment from MMC... OK  
In: serial  
Out: serial  
Err: serial  
Net: eth0: ethernet@5800a000  
lcd_id = 07  
Hit any key to stop autoboot: 0  
STM32MP> mm 0x50000218  
50000218: 00030000 ? 0x00030700  
5000021c: 00030700 ? q  
STM32MP> mii info 0x4  
PHY 0x04: OUI = 0x0000, Model = 0x10, Rev = 0x0A, 10baseT, HDX  
STM32MP>
```

图 3-29 mii info 命令结果

mii info 用于读取 PHY 芯片相关信息，是面向网络物理层的底层调试命令。该命令的加入，使本次实验不再仅限于启动命令本身，而是扩展到了网络相关底层状态的识别。对于后续涉及网络下载镜像、网络启动和 PHY 调试的场景，这条命令具备很强的延伸价值。

四、实验现象归纳

4.1 命令链条已经形成完整前后关系

从实验记录整体来看，命令不是孤立执行的，而是按层次逐步推进。先通过查询类命令建立对平台状态的认识，再通过环境变量命令介入系统配置，再通过 DRAM 命令验证数据操作能力，之后进入块设备与文件系统，再进入启动命令验证，最后扩展到维护与网络调试命令。这样的顺序使实验内容形成了清晰的任务链，而不是零散的命令堆积。

4.2 变量、内存、文件系统和启动之间存在直接衔接

环境变量决定默认策略，内存命令验证数据状态，文件系统命令负责搬运镜像，启动命令负责最终移交控制权。实验过程中这几类命令并不是并列关系，而是顺序衔接关系。只要其中某一环失效，后续现象就会发生变化。因此，对 U-Boot 的理解应建立在链条思维上，而不是建立在单条命令记忆上。

4.3 U-Boot 的作用不止于“拉起内核”

从 ums、run、mii info 的表现可以看出，U-Boot 不只是用来启动 Linux 的短暂过渡程序。它同时承担了维护、脚本组织和底层设备诊断的角色。这样理解 U-Boot，更符合嵌入式开发中的真实使用场景。

五、实验中暴露出的关键问题

5.1 修改之后必须验证，而不能只看命令是否执行

环境变量需要在重启后通过倒计时变化验证，内存写入需要通过回读验证，文件写入需要通过再次查询验证。底层实验最忌讳的思路，就是把“命令已经输入并返回”直接等同于“结果已经正确生效”。本次实验中，几乎每一类关键操作都体现了“写入后再验证”的必要性。

5.2 地址、长度和对象单位必须同时理解

内存命令和块设备命令最容易出问题的地方，并不只是语法，而是参数解释。长度到底是对象个数还是总字节数，地址是否处于合理范围，数值是否按

十六进制理解，这些都决定了命令结果是否可信。只会抄命令格式而不理解参数意义，实验很容易流于表面。

5.3 内核已启动并不等于系统已经完全启动成功

只要看到了 Linux 内核日志，就说明 U-Boot 阶段已经完成主要引导任务。若此后出现根文件系统相关报错，应把分析重点转向 Linux 阶段，而不是把所有问题都重新归到 U-Boot 启动命令上。区分不同阶段故障，是学习启动流程时必须建立的判断能力。

5.4 底层命令常常依赖硬件前置条件

mii info 之前需要先保证 ETHMAC 时钟可用，ums 在不同主机系统上会表现出不同可用性，串口日志是否正常又依赖终端参数匹配。这些现象表明，底层命令不仅有语法条件，还有硬件状态条件。理解这种双重条件，才能避免把所有失败都误判为“命令用错了”。

六、实验结论

本次 U-Boot 命令实验已经覆盖了交互环境建立、查询命令、环境变量管理、DRAM 操作、EMMC 与 ext4 访问、Linux 启动验证以及扩展调试命令等多个任务块。实验现象表明，开发板能够稳定进入 U-Boot 命令行；查询类命令能够正确输出状态信息；环境变量的修改、保存、新建和删除已经形成完整验证链；DRAM 可实现读、写、填充、复制和比较；EMMC 与 ext4 文件系统中的镜像文件能够被查询、读取和写回；Linux 内核能够通过 bootm 从 EMMC 启动；bootcmd 能够固化默认启动过程；ums、run、mii info 则进一步体现了 U-Boot 在维护和底层调试方面的作用。

从课程学习意义上看，本次实验的价值不在于记住若干条命令本身，而在于建立一条完整的 U-Boot 分析路径：先看平台状态，再看系统配置；先确认镜像是否存在，再确认镜像是否进入 DRAM；先判断 U-Boot 是否已把内核拉起，再分析 Linux 后续阶段是否存在根文件系统问题；先检查命令语法，再检查硬件前置条件是否满足。只有形成这样的链式理解，U-Boot 命令学习才真正具备长期价值。